Secure Distributed State Management for Stateful Signatures with a Practical and Universally Composable Protocol

Johannes Blömer
Paderborn University
bloemer@upb.de

Henrik Bröcher
Paderborn University
henrik.broecher@upb.de

Volker Krummel
Utimaco IS GmbH
volker.krummel@utimaco.com

Laurens Porzenheim
Paderborn University
laurens.porzenheim@upb.de

***Abstract*—Stateful signatures like the NIST standardized signature schemes LMS and XMSS provide an efficient and mature realization of post-quantum secure signature schemes. They are recommended for long-term use cases like e.g. firmware signing. However, stateful signature schemes require to properly manage a so-called state. In stateful signature schemes like LMS and XMSS, signing keys consist of a set of keys of a one-time signature scheme and it has to be guaranteed that each one-time key is used only once. This is done by updating a state in each signature computation, basically recording which one-time keys have already been used. While this is straightforward in centralized systems, in distributed systems like secure enclaves consisting of e.g. multiple hardware security modules (HSMs) with limited communication keeping a distributed state that at any point in time is consistent among all parties involved presents a challenge. This challenge is not addressed by the current standardization processes.**

In this paper we present a security model for the distributed key management of post-quantum secure stateful signatures like XMSS and LMS. We also present a simple, efficient, and easy to implement protocol proven secure in this security model, i.e. the protocol guarantees at any point in time a consistent state among the parties in a distributed system, like a distributed security enclave. The security model is defined in the universal composabilty (UC) framework by Ran Canetti by providing an ideal functionality for the distributed key management for stateful signatures. Hence our protocol remains secure even if arbitrarily composed with other instances of the same or other protocols, a necessity for the security of distributed key management protocols. Our main application are security enclaves consisting of HSMs, but the model and the protocol can easily be adapted to other scenarios of distributed key management of stateful signature schemes.

***Index Terms*—distributed state, hash-based signature, stateful hash-based signature, universal composability, secure enclave**

This work has received funding from the German Federal Ministry of Research, Technology and Space (BMFTR) within the PhoQuant project (Grant No. 13N16103).

1. Introduction

Our current public key infrastructure is insecure against quantum computers. This necessitates a transition from today's classical, number-theoretic systems (like RSA and ECC) to quantum-resistant (post-quantum) algorithms that can withstand attacks from quantum computers. Hence, worldwide standardization organizations select appropriate post-quantum secure cryptographic algorithms to be used in the near future. Among these algorithms and primitives are stateful hash-based signature schemes like the Leighton-Micali-Signatures (LMS) and the extended Merkel Signature Scheme (XMSS). Unlike stateless schemes such as ML-DSA, RSA or ECDSA, stateful hash-based signature schemes maintain an internal state that must be updated after each signature to ensure security. While not intended for general use, stateful hash-based signature schemes have been recommended by the NIST for certain use cases. In addition to being post-quantum secure, stateful hash-based signatures offer several desirable features: they have strong, long-term, and well-understood security, stemming from the security of hash functions that have been studied intensively for decades; they are easy to implement; they are forward secure; signature generation and verification are efficient, especially verification can be performed on constrained devices. These properties make them attractive for example for software authentication on constrained devices, with the main disadvantage of stateful signatures being the necessity for keeping a state.

In stateful hash-based signature schemes like LMS and XMSS, signing keys consist of a set of keys of a one-time signature scheme (OTS), e.g. the Lamport-Diffie OTS [12] or the Winternitz OTS [7], and it has to be guaranteed that each one-time key is used only once, which is done by keeping the state, basically recording which one-time keys have already been used. Although great care has been taken by standardization algorithms to select post-quantum cryptographic algorithms like stateful hash-based signatures together with appropriate realizations and parameters, for stateful signatures only general recommendations about the state handling are provided to ensure that keys are used only once, i.e., state handling should be done with extreme care in highly controlled environments. While this may be

sufficient for centralized systems, distributed systems like security enclaves consisting of e.g. multiple hardware security modules (HSMs) introduce additional challenges, as communication over possibly insecure, limited channels is required to maintain a state that is consistent among all modules involved. Hence there is an obvious demand for distributed protocols for the state handling of stateful signatures, whose security is well-defined and is based on few and clearly defined assumptions on the system, that can be used in combination with a variety of stateful signature schemes, that require only limited communication between the parties of the distributed system, and that can be efficiently implemented.

Our contribution. In this paper, we provide the first distributed state management protocol for stateful hash-based signatures, e.g. LMS and XMSS, the stateful hash-based signatures recommend by NIST. Our protocol is simple and practical, it covers all phases of state management and signing: the key generation, the key distribution, the key assignment and the signing phases. The protocol can be used with any stateful hash-based signature scheme used in practice, and its security is shown in the universal composability framework. Universal Composability (UC) is a framework in cryptography, introduced by Ran Canetti [4], that provides a rigorous way to analyze and prove the security of cryptographic protocols when they are composed with other protocols. UC security ensures that a protocol remains secure even when it runs concurrently with other (potentially arbitrary) protocols in a complex environment. Hence it offers exactly the type of security necessary for a distributed state handling protocol for stateful hash-based signature schemes. To show that our protocol is secure in the UC framework, we first define what security means for a protocol of this kind, which we do by defining a so-called functionality, which is of independent interest for the integration of stateful hash-based signature into higher-level protocols. In Section 3 we provide a more in-depth roadmap to our main result without going into the often technical and tedious details of the UC framework.

Related work. Stateful hash-based signature schemes have a long history, going back to the constructions of one-time signatures by Lamport in 1979 and the introduction of hash trees by Merkle (Merkle trees) in 1987 to compress the key sizes of signature schemes based on one-time signatures. This history eventually led to schemes like LMS [13, 14], introduced in 1997, and XMSS, introduced in 2011 in [2], and their hierarchical extensions HMS and XMSS+, respectively. Since hash-based signature schemes only use hash functions as cryptographic primitives, they are considered to be mature post-quantum secure signature schemes. Consequently, already in 2020 LMS, XMSS and their hierarchical extensions were standardized as post-quantum signatures schemes by the NIST and the IETF, preceding the standardization of stateless signatures schemes by four years. Due to their statefulness, hash-based signatures are not considered as generally applicable signatures schemes, their use is recommended only in certain circumstances. An important use case for stateful hash-based signatures schemes is the authentication of firmware updates for constrained devices (see [6]).

State handling, although a critical issue for the secure application of hash-based signatures schemes, so far has received little attention in the scientific community. The most extensive research into this issue has been done in [15], see also the recommendations in [6]. But [15] only considers the case where one entity (equipped with non-volatile and volatile memory) is responsible for the computation of signatures and the state handling. The case of distributed state handling, to the best of our knowledge, has not been studied systematically. In particular, no distributed protocols for state handling with security proofs exist.

Universal composability, introduced by Ran Canneti in 2001, although the journal paper only appeared in 2020 [4], is a versatile technique for proving the security of cryptographic primitives, distributed or otherwise. Its main advantage compared to other security frameworks (e.g. experiment-based security, Dolev-Yao) lies in its composition theorem: UC-secure protocols can be composed with each other to yield complex but still UC-secure protocols. The UC framework has been used extensively in cryptography. For numerous cryptographic tasks UC-secure protocols have been designed, by first describing ideal functionalities for the task and subsequently proving concrete protocols to securely realize the corresponding functionality (for a brief introduction to the UC framework see Section 2.4). Examples include symmetric and asymmetric encryption, message authentication codes, commitment schemes, zero-knowledge proofs, and, most importantly for this work, signature schemes. Alternatives to the UC framework have been proposed. They follow the same blueprint as Canetti's initial framework, but try to simplify it in various ways (see for example [10, 17]). In this paper we use the original UC framework as the most mature and popular framework.

We are not aware of previous formalizations of stateful (hash-based) signatures, with or without state handling, in the UC framework, or any other comparable framework. On the other hand, the formalization and realization of stateless signature schemes in the UC framework has a long and surprisingly complex history. In this paper, we build on the insights and results that emerged in the last twenty years of research on UC-secure signature schemes. This research is well summarized in [5], from first generation ideal functionalities through second and third generation functionalities and, finally, the fourth generation functionality presented in [5], using the terminology of [5]. As argued in [5], although ideal functionalities from the first through the third generation provide unforgeability of signatures, they do not provide so-called *unstoppability* (cf. Section 4), which is helpful for using a functionality in higher-level protocols. We build on the results in [5] to present an ideal functionality for stateful (hash-based) sig-

natures schemes with distributed state handling, that guarantees all the properties of unstoppable signatures in addition to unforgeabilty. As a building block to our protocol, we also build on a functionalities for retrieving pre-shared keys. In [11] they describe such a functionality for a key that is pre-shared between two parties, which we modify to allow groups of arbitrary many parties.

Organization. In Section 2.1 we present all relevant details on stateful hash-based signatures. In Section 2.2 we introduce signatures with subkeys as an abstraction of typical hash-based signatures like LMS and XMSS. In Section 2.3 we define authenticated encryption with associated data and its security. In Section 2.4 we briefly introduce the UC framework. To guide the reader through the often tedious details of security definitions and proofs in the UC framework, in Section 3 we provide an introduction to our main results and the major steps in the security proofs. In Section 4 we state our base assumption and model it as a functionality. We then define and discuss the ideal functionality for distributed state management for signatures with subkeys. In Section 5 we present our protocol for the distributed state management in signature schemes with subkeys and argue that it realizes the ideal functionality for distributed state management. In Section 6 we discuss several extensions to our ideal functionality and the corresponding extensions to the main protocol. In Appendix A we describe two ideal functionalities and protocols realizing them which we use in the proof of our main result about the security of the protocol for distributed state management. In Appendix B we give the full proof for our main theorem.

2. Preliminaries

By $[n]$ we denote the set of natural numbers from 1 to n. Key words, which we use to indicate what a message is for, are written in monospace font, such as `init`. We use the operator $a \leftarrow b$ to indicate assigning value b to variable a. If A is a set and $\oplus$ is a set operator, we denote by $A \xleftarrow{\oplus} b$ the operation $A \leftarrow A \oplus b$. In case b is a distribution (e.g. induced by the output of a probabilistic algorithm) the operation $a \xleftarrow{\$} b$ denotes drawing a random value according to b and assigning it to a. If B is a set $a \xleftarrow{\$} B$ denotes drawing uniformly at random from B and assigning that value to a. We generally use $\perp$ as an error symbol, while $\top$ denotes an empty message. If in a functionality or protocol we say to output $\perp$, we mean sending $\perp$ to the caller of the interface. This is to improve readability.

2.1. Stateful Hash-Based Signatures

As mentioned in the introduction, stateful hash-based signature schemes like LMS and XMSS typically follow a similar pattern. A signing key consists of multiple signing keys of a one-time signature scheme such as Lamport or Winternitz one-time signatures. The public keys of the one-time signature scheme are then hashed into a Merkle tree, as can be seen in Figure 1, where the hash at the root node forms the public key of the hash-based signature scheme.

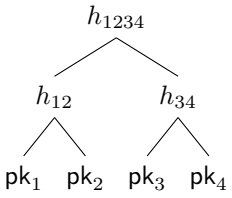

Figure 1. The typical pattern of a hash-based signature. One-time signature keys are hashed into a Merkle tree.

Whenever one wants to generate a signature with the hash-based signature scheme, one uses a one-time signing key to sign the message, collects all sibling nodes of the path from the one-time public key to the root, and appends these to signature. To verify the signature, one first verifies the one-time signature, and then checks whether the one-time public key belongs to the Merkle tree defined by the root, which is done by partially recomputing the Merkle tree with the sibling nodes up to the root.

It is important that every one-time key is used only once. Stateful hash-based schemes solve this restriction by remembering which one-time key was used in the state, while stateless schemes employ other strategies. Different hash-based schemes then expand upon this idea by, e.g., replacing the Merkle tree with more complex structures, such as a multi-tree. However, for our case it is only important that there exist one-time keys and one-time signatures which can be verified to belong to a hash-based signature scheme by some algorithm.

2.2. Signature Schemes with Subkeys

To express the general structure of a stateful hash-based signature scheme in a formal way we introduce *signatures with subkeys*. The syntax of signatures with subkeys follows the syntax of usual hash-based signature schemes like LMS and XMSS. However, instead of computing just one secret key for a public key, a signature with subkeys computes several keys for a public key.

Definition 2.1. *A* signature scheme with subkeys *(SwS) consists of two probabilistic polynomial time (ppt) algorithms* $\mathsf{KeyGen}, \mathsf{Sign}$, *and a deterministic polynomial time algorithm* Vrfy*:*

- $\mathsf{KeyGen}(1^\lambda) \xrightarrow{\$} ((\mathsf{sk}_i)_{i\in[\ell_{sub}]}, \mathsf{pk})$*: On input a security parameter* 1^λ*, the key generation algorithm outputs a vector of* ℓ_{sub} *subkeys* $(\mathsf{sk}_i)_{i\in[\ell_{sub}]}$ *and a public key* pk*.*
- $\mathsf{Sign}(1^\lambda, \mathsf{sk}_i, \mu) \xrightarrow{\$} \sigma$*: On input security parameter* 1^λ*, a subkey* sk_i *and a message* μ*, the signing algorithm outputs a signature* σ*.*
- $\mathsf{Vrfy}(1^\lambda, \mathsf{pk}, \mu, \sigma) \xrightarrow{\$} b$*: On input security parameter* 1^λ*, a public key* pk*, a message* μ *and a*

signature σ, the verification algorithm outputs a bit b.

We say that the signature scheme is correct, if for all λ, all $(\mathsf{sk}_i)_{i\in[\ell_{sub}]}$ output by $\mathsf{KeyGen}(1^\lambda)$, all $j \in [\ell_{sub}]$, all messages μ, and all σ output by $\mathsf{Sign}(1^\lambda, \mathsf{sk}_j, \mu)$ we have that $\mathsf{Vrfy}(\mathsf{pk}, \mu, \sigma) = 1$. We assume there is a function $\mathcal{L}$ mapping index i and public key pk to the subkey length $|\mathsf{sk}_i|$.

Syntactically, stateful hash-based signature schemes like LMS and XMSS are signatures with subkeys and their security transfers.

Definition 2.2. *We say that a signature scheme with subkeys is strong EUF-CMA one-time secure (or secure), if there exists a negligible function* negl *such that for all adversaries $\mathcal{A}$ we have*

$$\Pr\Big[\mathsf{Vrfy}(1^\lambda, \mathsf{pk}, \mu^*, \sigma^*) = 1 \wedge (\cdot, \mu^*, \sigma^*) \notin \mathcal{Q} : ((\mathsf{sk}_i)_{i\in[\ell_{sub}]}, \mathsf{pk}) \xleftarrow{\$} \mathsf{KeyGen}(1^\lambda), (\mu^*, \sigma^*) \xleftarrow{\$} \mathcal{A}^{\mathsf{SigO}(\cdot,\cdot)}(\mathsf{pk})\Big] \leq \mathsf{negl}(\lambda),$$

where $\mathcal{Q}$ is an initially empty set and $\mathsf{SigO}(j, \mu)$ outputs $\perp$ if $j \notin [\ell_{sub}]$ or $(j, \cdot, \cdot) \in \mathcal{Q}$, else it outputs $\sigma \xleftarrow{\$} \mathsf{Sign}(1^\lambda, \mathsf{sk}_j, \mu)$ and adds (j, μ, σ) to $\mathcal{Q}$.

The signing oracle allows each subkey to be used *only once*. Hence this security notion of signatures with subkeys conforms to the standard security notions for stateful hash-based signatures, i.e. they are unforgeable if every subkey is used at most once. Consequently, in order for an application to use a SwS securely, it must make sure to *never* call the signing algorithm with the same subkey twice.

We also require that an SwS scheme is *linear time*, which states that Sign and Vrfy run in time linear in their input length and polynomial in the security parameter.

Definition 2.3 (Linear-time)**.** *A signature scheme with subkeys $\Sigma = (\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Vrfy})$ is linear time if there exists a polynomial T such that $\mathsf{Sign}(1^\lambda, \mathsf{sk}_i, \mu)$ and $\mathsf{Vrfy}(1^\lambda, \mathsf{pk}, \mu, \sigma)$ always halts within $T(\lambda) \cdot (\ell + 1)$ steps, where ℓ is the total length of the inputs.*

This is merely a technical requirement for our security proof as in [5] and schemes in practice typically fulfill this requirement.

By defining the syntax of an SwS scheme to output all subkeys in KeyGen, we chose to stay as general as possible. Another possibility, for example, would have been to require KeyGen to output a seed for a pseudo-random generator with which one can generate all subkeys. Our choice carries some design implications for our protocol and security definition. We discuss a different syntax and its design implications in Section 6.

2.3. Authenticated Encryption with Associated Data

Due to the distributed nature of our protocol, we need communication between parties. We need authenticated, confidential communication, which we enable through *authenticated encryption with associated data*. The following definitions are taken from [16], however, we remove the nonce and instead use a probabilistic encryption algorithm.

Definition 2.4. *An authenticated encryption scheme with associated data (AEAD) is a tuple of three ppt polynomial time algorithms* $(\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec})$.

- $\mathsf{KeyGen}(1^\lambda) \to k$*: The key generation algorithm on input a security parameter 1^λ outputs a key k.*
- $\mathsf{Enc}(k, h, \mu) \to c$*: The encryption algorithm on input a key k, a header $h \subseteq \{0,1\}^*$, and a message $\mu \subseteq \{0,1\}^*$ outputs a ciphertext c.*
- $\mathsf{Dec}(k, h, c) \to \mu$*: The decryption algorithm on input a key k, a header h, and a ciphertext c outputs a message μ.*

We assume that both the header space and message space have a linear-time membership test and that some μ being in the message space implies all other messages of the same length also belonging to the message space.

The following CPA security definition is slightly changed from [16] to better suit our needs.

Definition 2.5. *We say that an AEAD scheme is CPA secure, if there exists a negligible function* negl *such that for all adversaries $\mathcal{A}$ we have that*

$$\left|\Pr\left[\mathcal{A}^{\mathsf{LoR}(0,\cdot,\cdot)}(1^\lambda) = 1 : k \xleftarrow{\$} \mathsf{KeyGen}(1^\lambda)\right] - \Pr\left[\mathcal{A}^{\mathsf{LoR}(1,\cdot,\cdot)}(1^\lambda) = 1 : k \xleftarrow{\$} \mathsf{KeyGen}(1^\lambda)\right]\right| \leq \mathsf{negl}(\lambda),$$

where the oracle $\mathsf{LoR}(b, (h_0, \mu_0), (h_1, \mu_1))$ returns $\mathsf{Enc}(k, h_b, \mu_b)$.

Definition 2.6. *We say that an AEAD scheme is unforgeable, if there exists a negligible function* negl *such that*

$$\Pr\Big[\mu \neq \perp \wedge (h, \mu, c) \notin \mathcal{Q} : k \xleftarrow{\$} \mathsf{KeyGen}(1^\lambda), (h, c) \xleftarrow{\$} \mathcal{A}^{\mathsf{EncO}(\cdot,\cdot,\cdot)}(1^\lambda), \mu \xleftarrow{\$} \mathsf{Dec}(k, h, c)\Big] \leq \mathsf{negl}(\lambda),$$

where $\mathcal{Q}$ is an initially empty set and $\mathsf{EncO}(h, \mu)$ computes $c \xleftarrow{\$} \mathsf{Enc}(k, h, \mu)$, adds (h, μ, c) to $\mathcal{Q}$, and outputs c.

2.4. The Universal Composability Framework

In this section we give a brief introduction to the universal composability framework (UC framework) by Ran Canetti. For more thorough and technical introductions we refer to [4]. In [5] a high-level introduction to universal composability is given. In addition, this works presents the state of the art for security of signatures in the UC framework. For

general introductions to the security of cryptographic protocols we refer to [9, 3].

In practice, cryptographic protocols rarely operate in isolation – they are composed with other protocols, often concurrently, within larger systems. Traditional security models (like stand-alone or sequential composition models) fail to guarantee security when protocols interact arbitrarily.

The Universal Composability (UC) framework, introduced by Ran Canetti (2001), addresses this problem by providing a general, modular, and composable definition of security. Its key goal is to ensure that if a protocol is proven secure in isolation, it remains secure when composed with other secure protocols – even under arbitrary concurrency and interleaving of messages.

The UC framework defines security via emulation of an ideal functionality. An ideal functionality $\mathcal{F}$ represents the desired task (e.g., key exchange, commitment, zero-knowledge proof) executed by a trusted third party that behaves perfectly. Parties $\mathcal{P}_i, i = 1, \ldots, n$ that want to jointly execute the task can interact with the ideal functionality. This setting is called the *ideal model*. A real protocol π is a concrete distributed protocol, jointly run by parties $\mathcal{P}_1, \ldots, \mathcal{P}_n$ that aims to implement $\mathcal{F}$ using secure channels, cryptographic primitives, etc. This is called the *real model*. Informally, a protocol π is UC-secure if no adversary $\mathcal{A}$ interacting with π can achieve anything that cannot also be achieved in the ideal model where all parties interact with $\mathcal{F}$ and a corresponding ideal adversary, called a simulator $\mathcal{S}$.

To define UC-security more formally, in addition to $\mathcal{F}$, the parties $\mathcal{P}_i$, $\mathcal{A}$ and $\mathcal{S}$, there is the so-called environment $\mathcal{E}$. The environment $\mathcal{E}$ represents everything external to the protocol (e.g., other concurrently running protocols, users, the network). It supplies inputs to the protocol participants, activates the parties $\mathcal{P}_i$, and observes their outputs. Importantly, $\mathcal{E}$ also interacts with the adversary $\mathcal{A}$ (in the real model) and the simulator $\mathcal{S}$ (in the ideal model). All these entities are modeled as *interactive Turing machines* (ITMs). In addition to the usual tapes, an ITM has communication tapes (read-only and write only) that allow it to interact and communicate with other ITMs. The admissible communication between parties $\mathcal{P}_1, \ldots, \mathcal{P}_n$, the environment $\mathcal{E}$, and the adversary $\mathcal{A}$ or simulator $\mathcal{S}$, respectively, is defined in subtle ways, leading to varying forms of UC-security (see the literature cited above). Usually, adversary $\mathcal{A}$ controls the communication between parties $\mathcal{P}_i$ by reading the messages exchanged and injecting messages; furthermore $\mathcal{A}$ may corrupt parties $\mathcal{P}_i$. In this paper, we assume that the adversary $\mathcal{A}$ controls communication but *cannot* corrupt parties $\mathcal{P}_i$. This is motivated by our main application, i.e. the parties responsible for distributed state management run in a security enclave. All Turing machines we consider are probabilistic TMs. Hence, they all have random tapes, equipping them with arbitrarily long strings of uniformly and independently distributed bits. By $\text{IDEAL}_{\mathcal{F},\mathcal{S},\mathcal{E}}(\kappa, z)$ denote $\mathcal{E}$'s output on input z and security parameter κ, after interacting with the simulator $\mathcal{S}$ and (dummy) parties $\mathcal{P}_1, \ldots, \mathcal{P}_n$, that interact with the ideal functionality $\mathcal{F}$. The output of $\text{IDEAL}_{\mathcal{F},\mathcal{S},\mathcal{E}}(\kappa, z)$ is a random variable, depending on the randomness of all entities involved. Similarly, by $\text{REAL}_{\pi,\mathcal{A},\mathcal{E}}(\kappa, z)$ denote $\mathcal{E}$'s output on input z and security parameter κ, after interacting with adversary $\mathcal{A}$ and parties $\mathcal{P}_1, \ldots, \mathcal{P}_n$, who run protocol π with input z. Equally to before, $\text{REAL}_{\pi,\mathcal{A},\mathcal{E}}(\kappa, z)$ is a random variable depending on the randomness of all entities involved. Finally, for ensembles of distributions $X = \{X_v\}_{v\in\{0,1\}^*}, Y = \{Y_v\}_{v\in\{0,1\}^*}$, we denote by $X \approx_c Y$ that the two ensembles are computationally indistinguishable and by $X \equiv Y$ that they are perfectly indistinguishable (see [8] for a formal definition).

Definition 2.7. *Protocol π realizes an ideal functionality $\mathcal{F}$, or is simply UC-secure, if for every probabilistic polynomial time adversary (ppt) $\mathcal{A}$ there exists a ppt simulator $\mathcal{S}$ such that for every ppt environment $\mathcal{E}$, the following two distribution ensembles are computationally indistinguishable.*

$$\{\text{REAL}_{\pi,\mathcal{A},\mathcal{E}}(\kappa, z)\}_{\kappa\in\mathbb{N},z\in\{0,1\}^*} \approx_c \{\text{IDEAL}_{\mathcal{F},\mathcal{S},\mathcal{E}}(\kappa, z)\}_{\kappa\in\mathbb{N},z\in\{0,1\}^*}$$

In practice, higher-level protocols often depend on other secure building blocks. Hybrid models and the important *composition theorem* allows us to treat subprotocols as if they were ideal functionalities during analysis without having to prove everything from scratch. To make this more precise, we restrict ourselves to hybrids with a single ideal functionality; the generalization to multiple functionalities is straightforward. Let $\mathcal{F}$ be an ideal functionality as above. The $\mathcal{F}$-hybrid model extends the real model with the ideal functionality $\mathcal{F}$. The parties communicate with each other in exactly the same way as in the real model described above; however, they can interact with (and activate) $\mathcal{F}$ as in the ideal model. There is also communication between the ideal functionality $\mathcal{F}$ and adversaries $\mathcal{A}$ via so-called back-door tapes. By $\text{HYBRID}^{\mathcal{F}}_{\pi,\mathcal{A},\mathcal{E}}(\kappa, z)$ denote $\mathcal{E}$'s out on input z and security parameter κ after interacting in a $\mathcal{F}$-hybrid model with adversary $\mathcal{A}$ and parties $\mathcal{P}_1, \ldots, \mathcal{P}_n$ who run protocol π. As before $\mathcal{E}$'s output is a random variable. Briefly, and somewhat informally, the composition theorem can be stated as follows.

Theorem 2.8 (UC Composition Theorem (Canetti))**.** *Let $\mathcal{E}$ be an ideal functionality and ρ be a protocol that realizes $\mathcal{F}$. Let π be a protocol in the $\mathcal{F}$-hybrid model and let π^ρ be the protocol identical to π, except when a party $\mathcal{P}$ activates $\mathcal{F}$, protocol π^ρ activates the corresponding interface of $\mathcal{P}$ in ρ. Then for every real model ppt adversary $\mathcal{A}$ there is ppt simulator $\mathcal{S}$ in the $\mathcal{F}$-hybrid model such that*

$$\{\text{REAL}_{\pi^\rho,\mathcal{A},\mathcal{E}}(\kappa, z)\}_{\kappa\in\mathbb{N},z\in\{0,1\}^*} \approx_c \{\text{HYBRID}^{\mathcal{F}}_{\pi,\mathcal{S},\mathcal{E}}(\kappa, z)\}_{\kappa\in\mathbb{N},z\in\{0,1\}^*}.$$

The composition theorem shows that UC-security is preserved under composition of UC-secure proto-

cols. This has two benefits. First, it allows to break down a complex system into simpler chunks, then analyze protocols for each of the chunks (as standalone), then deduce the security of the original, composite system. This will be used extensively in the proofs in this paper. Second, preservation of UC-security guarantees security when the protocol is running alongside potentially unknown protocols that may even be designed later, assuming all protocols involved are proven to be UC-secure. This is a desirable property for distributed state management protocols such as the one we present in this work.

3. Roadmap

In the next sections we provide a comprehensive protocol for the distributed use of signatures with subkeys, and prove its security in the UC framework. Here we give an outline and roadmap for the steps necessary for this, informally state our assumptions and security goals and introduce the main building blocks for the protocol.

The protocol is run between a set of identical secure cryptographic processors (SCP) [1], e.g. HSMs or trusted platform modules, which we simply call parties. There is no hierarchy between parties. We do, however, assume that the parties are incorruptible. By this we mean that an SCP can be viewed as the realization of a cryptographic black box as it is used in many cryptographic security models. To be more precise, an SCP can store keys, execute cryptographic operations, execute maintenance operations like generate key, delete key in a reliable way and fulfill security requirements regarding confidentiality and authenticity without leaking key information to the outside. Furthermore, the SCPs can store data permanently and protect the authenticity of stored data, e.g., key information or state information. We do not assume that there is an infrastructure in place that allows the parties to communicate securely, i.e. secure channels that guarantee confidentiality and authenticity. We assume communication is under adversarial control, hence messages can be dropped arbitrarily. Our informal security requirements are twofold. First, the protocol should guarantee that signatures are unforgeable in the standard sense of existential unforgeability under chosen message attacks (EUF-CMA), although we give signing authority to different parties and allow sending key material and permissions to use subkeys over an insecure network. Furthermore, we require that the protocol is secure in composition with other protocols in the UC framework.

To achieve these goals, our protocol π_{dsm} has three modes or phases:

1) key generation, in this mode secret keys and a public key for a signature scheme with subkeys are generated,
2) key and state distribution, in this mode key and state information is distributed among the parties involved in the protocol to ensure that each secret keys is used at most once, i.e. the parties involved have consistent views of the overall state,
3) signing (and verifying), in this mode signatures are computed (and if necessary, verified).

The protocol is described in Section 5. Its key ingredients are a secure authenticated encryption with associated data scheme (AEAD), to establish secure communication, and a signature scheme with subkeys for signing and verifying. In the protocol, we assume that in the beginning one SCP or party, generates the public key and all subkeys, which are then distributed to all other SCPs. Usage of subkeys is organized via responsibilities for keys. Initially, the first party holds responsibility for all subkeys but may then delegate responsibility for certain subkeys to other parties. Apart from this initialization step, the first party does not play a special role. In particular, the other parties may further delegate received responsibilities. Having only one party create the keys for the signature scheme simplifies the design of the protocol and, in practice, yields flexibility and simplicity.

To show that our protocol for distributed state management, which we call π_{dsm}, satisfies our requirements in the UC framework, we follow the usual blueprint to establish such results. We first describe a so-called ideal functionality $\mathcal{F}_{\text{dsm}}$ for signature schemes with distributed state management. This ideal functionality describes the desired properties of a secure signature scheme with distributed state management in the form of a trusted third party. In particular, the ideal functionality computes and verifies signatures. It keeps the complete state of the used SwS, which is not possible for the parties, thereby ensuring no subkey is used twice. Additionally, it provides adversaries with information and capabilities according to our adversarial model. Finally, it provides unstoppability, which is a property helpful for using the functionality in a higher-level protocol (cf. Section 4). The ideal functionality is given in Section 4. For the security proof, we give a proof overview in Section 5 and show formally in Section B that any adversary against the protocol in the real model can be simulated by a simulator against the ideal functionality in the ideal model such that both behave indistinguishably, proving that an adversary against the protocol can not learn or achieve more than the adversary against the ideal functionality, i.e. basically nothing by virtue of the design of the ideal functionality.

This brief description hides and brushes over many technical details, inherent to all UC security definitions and proofs. The use of simulation techniques to prove UC security typically renders ideal functionalities much more complex than its informal description as acting like a trusted third party suggests. This is certainly the case for the functionality $\mathcal{F}_{\text{dsm}}$. Therefore in Section 4 we explain in detail the rationale behind the ideal functionality, in addition to its formal definition. On the positive side, the Composition Theorem 2.8 not only shows that UC security is closed under composition, it also allows one to modularize security proofs by simulation.

In our case, in the simulation proof for protocol π_{dsm} we assume that instead of a real protocol to real-

ize secure communication, we have access to an ideal functionality for ideal communication. This leads to Theorem 5.1 and its proof. In a subsequent step, we define a real protocol for secure communication in Section A.2, and show that it securely realizes the ideal functionality for secure communication, again using the composition theorem and a more basic communication functionality as well as its secure realization in Section A.1.

4. Distributed State Management Functionality

Before formalizing the functionality for distributed state management, we first a brief overview over the real world protocol that we envision, and explain the security goals for it. We use the description and the goals to formalize them into a description of a trusted party or ideal functionality.

The protocol starts with a group of SCPs $\mathcal{H}$. One of the SCPs, let it be $\mathcal{P}_j$, sets up a new instance of an SwS scheme by generating keys with the KeyGen algorithm of the SwS scheme. Additionally, $\mathcal{P}_j$ assigns all generated subkeys to himself, meaning he has permission to use them. If he wants to distribute subkeys to some other party $\mathcal{P}_t \in \mathcal{H}$, he does so in two steps. First, he sends the public and secret keys to $\mathcal{P}_t$. We call this *notifying* the other party, i.e. after this step $\mathcal{P}_t$ is notified. Only after that does $\mathcal{P}_j$ unassign some subkeys from himself, and sends the permission for them to $\mathcal{P}_t$ so that they get assigned to him. Then, if a party was sent the public and secret keys and it has some subkeys assigned, it can use one assigned subkey to create a signature by invoking the Sign algorithm of the SwS scheme with this subkey. Signature verification works by simply invoking the Vrfy algorithm of the SwS scheme.

For security, one goal is the typical unforgeabilty guarantee of a signature scheme. In order to achieve this, the subkeys must stay secret from unauthorized parties when they are sent to authorized parties. Furthermore, it is important to ensure no subkey gets used twice, else the security condition of the SwS scheme that is used as a building block is violated. For this, for each subkey it is tracked to which party it is assigned, whether it is currently in transit, or whether was already used. It is important that for each subkey only *exactly one* of these three conditions is true to prevent double usage of a subkey. We call this property *subkey partition*. Another important aspect is that if key permission is sent to another party, the same message is never used twice to assign subkeys. Else one could send a subkey, assign it, use it, assign it again, and then use it a second time, violating the *subkey partition* property.

We can now start to get more formal and talk about defining the functionality $\mathcal{F}_{\text{dsm}}$. To model the unforgeability requirement, we take inspiration from [5]. In their modeling of a (standard) signature functionality, they provide the interfaces **Init**, **KeyGen**, **Sign**, and **Verify**, which we model similarly in out functionality. In **Init** we initialize the functionality (not the signature scheme) and set up some counters and sets, where most of them are used for a so-called *random-mode*. We explain this later in more detail. To set up the functionality, the ideal adversary is asked to provide algorithms $(\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Vrfy})$ in the **Init** interface, which are saved and later used in other interfaces of the functionality. It is important to note that the ideal adversary is *not* required to provide algorithms that form a secure SwS scheme. Instead, we only require that they are probabilistic algorithms. Furthermore, we check that they are linear time as defined in Definition 2.3 by aborting each execution of these algorithms if they take too long.

When a party $\mathcal{P}_j$ interacts with the **Key Generation** interface, the functionality generates keys $((\mathsf{sk}_i)_{i\in[\ell_{\text{sub}}]}, \mathsf{pk})$ with the adversarially provided KeyGen. It creates flags $\mathcal{N}_i^{\mathsf{pk}}$ that store whether a party $\mathcal{P}_i$ is notified for public key pk. $\mathcal{F}_{\text{dsm}}$ then stores that $\mathcal{P}_j$ is notified by setting $\mathcal{N}_j^{\mathsf{pk}} \leftarrow 1$, while all other parties are not notified by $\mathcal{N}_t^{\mathsf{pk}} \leftarrow 0, t \neq j$. Then, $\mathcal{F}_{\text{dsm}}$ creates sets $\mathcal{B}_i^{\mathsf{pk}}$, indicating which subkeys of public key pk are currently assigned to which party $\mathcal{P}_i$. $\mathcal{F}_{\text{dsm}}$ assigns all subkeys to $\mathcal{P}_j$ by setting $\mathcal{B}_j^{\mathsf{pk}} \leftarrow [\ell_{\text{sub}}^{\mathsf{pk}}]$, where $\ell_{\text{sub}}^{\mathsf{pk}}$ is the number of subkeys output by KeyGen. It also defines $\mathcal{B}_t^{\mathsf{pk}} \leftarrow \emptyset, t \neq j$ to indicate that other parties do not have permission to use subkeys, although they may know them if they are notified. We stress that the variables $\mathcal{N}_j^{\mathsf{pk}}, \mathcal{B}_j^{\mathsf{pk}} \leftarrow [\ell_{\text{sub}}^{\mathsf{pk}}], \ell_{\text{sub}}^{\mathsf{pk}}$ are defined per pk. Lastly, $\mathcal{F}_{\text{dsm}}$ stores that it generated pk itself.

When a party $\mathcal{P}_j$ interacts with the **Sign** interface, specifying a subkey by its index l, $\mathcal{F}_{\text{dsm}}$ checks thats $\mathcal{P}_j$ is notified and has permission to use subkey l. If so, the subkey is removed from $\mathcal{B}_j^{\mathsf{pk}}$ and $\mathcal{F}_{\text{dsm}}$ uses the adversarially provided Sign to generate some signature string. Then, $\mathcal{F}_{\text{dsm}}$ stores that it has generated the signature itself and outputs it to $\mathcal{P}_j$.

When a party interacts with the **Verify** interface, $\mathcal{F}_{\text{dsm}}$ first checks whether it if received the same input before. If so, it answers as before. This is to ensure consistency of verification. If the input has not been seen before, and if the public key and the signature have been generated by $\mathcal{F}_{\text{dsm}}$, it marks the signature as valid and answers accordingly. This is because if $\mathcal{F}_{\text{dsm}}$ has generated the public key and signature, the signature should be valid. (In the definition of $\mathcal{F}_{\text{dsm}}$ this is included in the previous check.) If only the public key was generated by $\mathcal{F}_{\text{dsm}}$, then it rejects the signature, since $\mathcal{F}_{\text{dsm}}$ should have exclusive power to create signatures under public keys it has generated. Else, i.e if the public key has not been generated by $\mathcal{F}_{\text{dsm}}$, then (and only then) $\mathcal{F}_{\text{dsm}}$ uses the adversarially provided Vrfy to decide whether the signature is valid or not. This is because in the case that a public key was not generated by $\mathcal{F}_{\text{dsm}}$, it does not care which signatures are valid or not, thus the adversary can decide via the definition of Vrfy.

In contrast to [5] we do not need corruption interfaces. This is because we assume that all SCPs and therefore parties remain honest. Thus, it is not necessary for us to, for example, store the randomness

used to generate a signature, which is needed in [5].

Another difference to [5] is that we introduce four new interfaces. These will be used to send and receive keys and subkey permissions. The first is **Notify**, which a party $\mathcal{P}_j$ can invoke to notify another party $\mathcal{P}_t$. However, $\mathcal{F}_{\text{dsm}}$ does not immediately set $\mathcal{N}_t^{\mathsf{pk}} \leftarrow 1$. Instead, the intention of $\mathcal{P}_j$ to notify $\mathcal{P}_t$ is told by $\mathcal{F}_{\text{dsm}}$ to the ideal adversary. Then, the ideal adversary can use the interface **Notified** to allow $\mathcal{F}_{\text{dsm}}$ to set $\mathcal{N}_t^{\mathsf{pk}} \leftarrow 1$.

This is similar for the interface **Assign**. A party $\mathcal{P}_j$ can invoke it to assign a set of subkeys req that they possess to another party $\mathcal{P}_t$. $\mathcal{F}_{\text{dsm}}$ unassigns subkeys req from $\mathcal{P}_j$ by updating $\mathcal{B}_j^{\mathsf{pk}}$, then tells the ideal adversary of $\mathcal{P}_j$'s intention. The ideal adversary can then use the **Assigned** interface to allow this transfer, after which $\mathcal{F}_{\text{dsm}}$ assigns req to $\mathcal{P}_t$ by updating $\mathcal{B}_t^{\mathsf{pk}}$. Here it is very important that $\mathcal{F}_{\text{dsm}}$ properly keeps track which subkeys are assigned to which party, in transit (i.e. the intention was sent to the ideal adversary, but he has not yet approved the transfer), or used. Assigned keys are in some $\mathcal{B}_j^{\mathsf{pk}}$, while keys in transit are stored in a set $\mathcal{K}_{\text{send}}$. Defined correctly, this ensures the *subkey partition* property.

We also want our functionality to be *unstoppable*, at least as much as possible. In [5] they introduce this notion of unstoppability, which means that a functionality is designed in a way that a protocol using such a functionality can expect the functionality to always work, regardless of what the ideal adversary does. For example, if the ideal adversary in **Init** supplies bad algorithms that always stop immediately, we cannot generate keys or signatures and would therefore have to output an error symbol. While this property is not interesting for the real world, as protocols are typically designed in a way that they cannot be stopped, unstoppability might aid or even enable proofs about indistinguishability in a *hybrid* model. It therefore is a desired property of a functionality. To prevent stopping of their functionality, [5] switch their functionality to a so-called *random mode* should an event happen that would prevent their functionality from working. If the functionality is in random mode, it behaves ideally, e.g. it generates public keys and signature uniformly at random while avoiding public keys and signatures seen before. Since all the operations they support, e.g. KeyGen, Sign, Vrfy, can be executed locally in the real world, they are able to define a fully unstoppable signature functionality. We, however, require communication over a network in the real world, since in our scenario we have multiple SCPs. Thus, we incorporate the random mode of [5] in the interfaces that in the real world are algorithms, i.e. (**Init**, **Key Generation**, **Sign**, **Verfiy**). For interfaces that in the real world need to use the network, i.e. (**Notify**, **Notified**, **Assign**, **Assigned**), we make no such guarantees. To enable switching to the random mode, $\mathcal{F}_{\text{dsm}}$ keeps track of all public keys and signatures it has seen by storing them in $\mathcal{K}_{\text{pk}}$ and $\mathcal{K}_{\text{sign}}$ respectively. It switches to random mode, if either the adversarially provided algorithms that take too long, if the same public key is generated for the second time, if the same signature is generated for a different message, if a signature is generated although it has been stored as a rejected signature (this would **Verify** to not be able to decide whether it should accept or reject this signature). Once $\mathcal{F}_{\text{dsm}}$ is in random mode, it will stay in random mode. Random mode then works as in [5] with the exception, that if **Key Generation** is called while $\mathcal{F}_{\text{dsm}}$ is in random mode, we set $\ell_{\text{sub}}^{\mathsf{pk}} \leftarrow \infty$. This is because we get the number of subkeys from the output of KeyGen, which is not executed in random mode.

It is important to note that this random mode is only a theoretic construct needed for higher-level protocols that use the $\mathcal{F}_{\text{dsm}}$ functionality in a hybrid way. For these protocols we ensure by the random mode that the functionality always works. However, in practice the random mode is never used. By the properties of the SwS scheme that we use in our protocol, the random mode cannot be triggered, thus we can ignore it in practice.

With this, we can formally define the functionality $\mathcal{F}_{\text{dsm}}$ for distributed state management.

Functionality $\mathcal{F}_{\text{dsm}}$

- Memory is per sid.
- Ignore any message from any party $\mathcal{P}$ that contains some session ID sid until party $\mathcal{P}$ sends $(\texttt{init}, \mathsf{sid})$.

Initialization On input $(\texttt{init}, \mathsf{sid})$ from party $\mathcal{P}$

1: If $(\texttt{init}, \mathsf{sid})$ is received for the first time, set $\ell_{\mathsf{pk}}, \ell_{\mathsf{sign}} \leftarrow 1$ and $\mathcal{K}_{\mathsf{pk}}, \mathcal{K}_{\mathsf{sign}}, \mathcal{K}_{\mathsf{send}} \leftarrow \emptyset$.
2: Send $(\texttt{init}, \mathsf{sid})$ to $\mathcal{S}$.
3: After first `init` with sid, upon *any* second message μ containing sid by some party $\mathcal{P}'$:
  a: If $\mathcal{P}' \neq \mathcal{S}$ or $\mu \neq (\texttt{algs}, \mathsf{sid}, \Sigma)$, where $(\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Vrfy}) := \Sigma$ is the description of three probabilistic Turing machines, set $\mathsf{rmode} = 1$.
  b: Else, set $\mathsf{rmode} = 0$, store $(\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Vrfy})$ and $s := |\Sigma|$.
4: If $\mathsf{rmode} = 1$, also process the second message using the interfaces below.

Key Generation On input $(\texttt{keygen}, \mathsf{sid}, \mathcal{H})$ from party $\mathcal{P}_j$

5: If $\mathcal{P}_j \notin \mathcal{H}$, output $\perp$.
6: If $\mathsf{rmode} = 0$, compute $((\mathsf{sk}_i)_{i \in [\ell_{\text{sub}}^{\mathsf{pk}}]}, \mathsf{pk}) \xleftarrow{\$} \mathsf{KeyGen}()$. If $\mathsf{pk} \in \mathcal{K}_{\mathsf{pk}}$ or KeyGen does not terminate in s steps, set $\mathsf{rmode} = 1$.
7: If $\mathsf{rmode} = 1$, sample $\mathsf{pk} \xleftarrow{\$} \{0,1\}^{\ell_{\mathsf{pk}}} \backslash \mathcal{K}_{\mathsf{pk}}$, set $\mathsf{sk} \leftarrow \perp$, and set $\ell_{\text{sub}}^{\mathsf{pk}} \leftarrow \infty$.
8: Set $\mathcal{N}_j^{\mathsf{pk}} \leftarrow 1$ and $\mathcal{B}_j^{\mathsf{pk}} \leftarrow [\ell_{\text{sub}}^{\mathsf{pk}}]$, and for $i \in \mathcal{H} \backslash \{\mathcal{P}_j\}$ set $\mathcal{B}_i^{\mathsf{pk}} \leftarrow \emptyset$ and $\mathcal{N}_i^{\mathsf{pk}} \leftarrow 0$.
9: Set $\mathcal{K}_{\mathsf{pk}} \xleftarrow{\cup} \{\mathsf{pk}\}$ and increment ℓ_{pk} until $\{0,1\}^{\ell_{\mathsf{pk}}} \backslash \mathcal{K}_{\mathsf{pk}} \neq \emptyset$.
10: Store $(\texttt{key}, \mathsf{sid}, \mathcal{H}, (\mathsf{sk}_i)_{i \in [\ell_{\text{sub}}^{\mathsf{pk}}]}, \mathsf{pk})$ in memory and send $(\texttt{pk}, \mathsf{sid}, \mathsf{pk}, \ell_{\text{sub}}^{\mathsf{pk}})$ to $\mathcal{P}_j$.

Notify On input $(\texttt{notify}, \mathsf{sid}, \mathsf{pk}, \mathcal{P}_t)$ from $\mathcal{P}_j$

11: Set $\mathcal{K}_{\text{pk}} \xleftarrow{\cup} \{\mathsf{pk}\}$ and increment ℓ_{pk} until $\{0,1\}^{\ell_{\text{pk}}} \setminus \mathcal{K}_{\text{pk}} \neq \emptyset$.
12: If $(\texttt{key}, \mathsf{sid}, \mathcal{H}, \cdot, \mathsf{pk})$ exists in memory for some $\mathcal{H}$, continue, else output $\perp$.
13: If $\mathcal{P}_j \notin \mathcal{H}$, $\mathcal{P}_t \notin \mathcal{H}$, or $\mathcal{N}_j^{\mathsf{pk}} = 0$, output $\perp$.
14: Store $(\texttt{notified}, \mathsf{sid}, \mathsf{pk}, \mathcal{P}_j, \mathcal{P}_t)$.
15: Send $(\texttt{notified}, \mathsf{sid}, \mathcal{H}, \ell_{\text{sub}}^{\mathsf{pk}}, \mathsf{pk}, \mathcal{P}_j, \mathcal{P}_t)$ to $\mathcal{S}$.

Notified On input $(\texttt{notified}, \mathsf{sid}, \mathsf{pk}, \mathcal{P}_j, \mathcal{P}_t)$ from $\mathcal{S}$

16: If $\mathcal{P}_t$ has not sent $(\texttt{init}, \mathsf{sid})$, output $\perp$.
17: Set $\mathcal{K}_{\text{pk}} \xleftarrow{\cup} \{\mathsf{pk}\}$ and increment ℓ_{pk} until $\{0,1\}^{\ell_{\text{pk}}} \setminus \mathcal{K}_{\text{pk}} \neq \emptyset$.
18: If $(\texttt{notified}, \mathsf{sid}, \mathsf{pk}, \mathcal{P}_j, \mathcal{P}_t)$ exists in memory, set $\mathcal{N}_t^{\mathsf{pk}} \leftarrow 1$.

Assign On input $(\texttt{assign}, \mathsf{sid}, \mathsf{pk}, \mathcal{P}_t, \mathsf{req})$ from party $\mathcal{P}_j$

19: Set $\mathcal{K}_{\text{pk}} \xleftarrow{\cup} \{\mathsf{pk}\}$ and increment ℓ_{pk} until $\{0,1\}^{\ell_{\text{pk}}} \setminus \mathcal{K}_{\text{pk}} \neq \emptyset$.
20: If $(\texttt{key}, \mathsf{sid}, \mathcal{H}, \mathsf{pk}, \cdot)$ exists in memory for some $\mathcal{H}$, continue, else output $\perp$.
21: If $\mathcal{P}_j \notin \mathcal{H}$ or $\mathcal{P}_t \notin \mathcal{H}$ output $\perp$.
22: If $\mathsf{req} \not\subseteq \mathcal{B}_j^{\mathsf{pk}}$ output $\perp$.
23: Set $\mathcal{B}_j^{\mathsf{pk}} \xleftarrow{\setminus} \mathsf{req}$.
24: Set $\mathcal{K}_{\text{send}} \xleftarrow{\cup} \{(\texttt{assgnd}, \mathsf{sid}, \mathsf{pk}, \mathcal{P}_j, \mathcal{P}_t, \mathsf{req})\}$
25: Send $(\texttt{assgnd}, \mathsf{sid}, \mathsf{pk}, \mathcal{P}_j, \mathcal{P}_t, \mathsf{req})$ to $\mathcal{S}$

Assigned On input $(\texttt{assgnd}, \mathsf{sid}, \mathsf{pk}, \mathcal{P}_j, \mathcal{P}_t, \mathsf{req})$ from $\mathcal{S}$

26: If $\mathcal{P}_t$ has not sent $(\texttt{init}, \mathsf{sid})$, output $\perp$.
27: Set $\mathcal{K}_{\text{pk}} \xleftarrow{\cup} \{\mathsf{pk}\}$ and increment ℓ_{pk} until $\{0,1\}^{\ell_{\text{pk}}} \setminus \mathcal{K}_{\text{pk}} \neq \emptyset$.
28: If $(\texttt{key}, \mathsf{sid}, \mathcal{H}, \cdot, \mathsf{pk})$ exists in memory for some $\mathcal{H}$, continue, else output $\perp$.
29: If $\mathcal{N}_t^{\mathsf{pk}} = 0$, or $\mathcal{P}_j \notin \mathcal{H}$ or $\mathcal{P}_t \notin \mathcal{H}$ output $\perp$.
30: If $(\texttt{assgnd}, \mathsf{sid}, \mathsf{pk}, \mathcal{P}_j, \mathcal{P}_t, \mathsf{req})$ exists in memory, continue, else output $\perp$.
31: Set $\mathcal{K}_{\text{send}} \xleftarrow{\setminus} \{(\texttt{assgnd}, \mathsf{sid}, \mathsf{pk}, \mathcal{P}_j, \mathcal{P}_t, \mathsf{req})\}$.
32: Set $\mathcal{B}_t^{\mathsf{pk}} \xleftarrow{\cup} \mathsf{req}$.
33: Send $(\texttt{keyset}, \mathsf{sid}, \mathsf{pk}, \mathcal{B}_t^{\mathsf{pk}})$ to $\mathcal{P}_t$.

Sign On input $(\texttt{sign}, \mathsf{sid}, \mathsf{pk}, \mu, l)$ from party $\mathcal{P}_j$

34: Set $\mathcal{K}_{\text{pk}} \xleftarrow{\cup} \{\mathsf{pk}\}$ and increment ℓ_{pk} until $\{0,1\}^{\ell_{\text{pk}}} \setminus \mathcal{K}_{\text{pk}} \neq \emptyset$.
35: If $(\texttt{key}, \mathsf{sid}, \mathcal{H}, \mathsf{sk}, \mathsf{pk})$ exists in memory for some $\mathcal{H}$, continue, else output $\perp$.
36: If $\mathcal{P}_j \notin \mathcal{H}$, output $\perp$.
37: If $\mathcal{N}_j^{\mathsf{pk}} = 0$, or $l \notin \mathcal{B}_j^{\mathsf{pk}}$, output $\perp$.
38: If $\mathsf{rmode} = 0$, parse $(\{0,1\}^*)^{\ell_{\text{sub}}} \leftarrow \mathsf{sk}$ and compute $\sigma \xleftarrow{\$} \mathsf{Sign}(\mathsf{sk}_l, \mu)$. If $(\texttt{sig}, \mathsf{sid}, \mathsf{pk}, \mu', \sigma)$ exists in memory for some $\mu' \neq \mu'$ or $(\texttt{badsig}, \mathsf{sid}, \mathsf{pk}, \mu, \sigma)$ exists in memory or Sign does not terminate in $(|\mu|+1) \cdot s$ steps, set $\mathsf{rmode} \leftarrow 1$.
39: If $\mathsf{rmode} = 1$, sample $\sigma \xleftarrow{\$} \{0,1\}^{\ell_{\text{sign}}} \setminus \mathcal{K}_{\text{sign}}$.
40: Set $\mathcal{B}_j^{\mathsf{pk}} \xleftarrow{\setminus} \{l\}$.
41: Set $\mathcal{K}_{\text{sign}} \xleftarrow{\cup} \{\sigma\}$ and increment ℓ_{sign} until $\{0,1\}^{\ell_{\text{sign}}} \setminus \mathcal{K}_{\text{sign}} \neq \emptyset$.
42: Store $(\texttt{sig}, \mathsf{sid}, \mathsf{pk}, \mu, \sigma)$ in memory and output $(\texttt{signature}, \mathsf{sid}, \mathsf{pk}, \mu, \sigma)$ to $\mathcal{P}_j$.

Verify On input $(\texttt{vrfy}, \mathsf{sid}, \mathsf{pk}, \mu, \sigma)$ from party $\mathcal{P}$

43: Set $\mathcal{K}_{\text{pk}} \xleftarrow{\cup} \{\mathsf{pk}\}$ and increment ℓ_{pk} until $\{0,1\}^{\ell_{\text{pk}}} \setminus \mathcal{K}_{\text{pk}} \neq \emptyset$.
44: If $(\texttt{sig}, \mathsf{sid}, \mathsf{pk}, \mu, \sigma)$ is in memory, set $b \leftarrow 1$.
45: Else, if $(\texttt{badsig}, \mathsf{sid}, \mathsf{pk}, \mu, \sigma)$ exists in memory, set $b \leftarrow 0$.
46: Else, if $(\texttt{key}, \mathsf{sid}, \cdot, \cdot, \mathsf{pk})$ exists in memory, set $b \leftarrow 0$.
47: Else, if $\mathsf{rmode} = 1$, set $b \leftarrow 0$.
48: Else, if $\mathsf{rmode} = 0$, set $b \xleftarrow{\$} \mathsf{Vrfy}(\mathsf{pk}, \mu, \sigma)$. If Vrfy does not finish in $(|\mu|+1) \cdot s$ steps, set $\mathsf{rmode} = 1$ and $b = 0$.
49: If $b = 0$, store $(\texttt{badsig}, \mathsf{sid}, \mathsf{pk}, \mu, \sigma)$ in memory.
50: If $b = 1$ and no $(\texttt{sig}, \mathsf{sid}, \mathsf{pk}, \mu, \sigma)$ exists in memory, store $(\texttt{sig}, \mathsf{sid}, \mathsf{pk}, \mu, \sigma)$ in memory.
51: Set $\mathcal{K}_{\text{sign}} \xleftarrow{\cup} \{\sigma\}$ and increment ℓ_{sign} until $\{0,1\}^{\ell_{\text{sign}}} \setminus \mathcal{K}_{\text{sign}} \neq \emptyset$.
52: Send $(\texttt{verified}, \mathsf{sid}, \mathsf{pk}, \mu, \sigma, b)$ to $\mathcal{P}$.

One might wonder why the interfaces **Notify** and **Notified** are necessary, and why **Assign** cannot be used directly. Before it makes sense for a party to accept subkey permission, it needs to know the subkeys. Since we want key generation to happen at a single party, this party needs to distribute the public key as well as the subkeys. It would be possible to transfer them together with **Assign** messages, however, since the adversary controls the network, we would need to send *all* subkeys and the public key in *every* **Assign** message. While possible, this would incur a noticeable overhead in practice, as we would need to encrypt all subkeys. For more discussion about alternatives, see Section 6.

5. DSM Protocol

With the functionality $\mathcal{F}_{\text{dsm}}$ in place, we continue with constructing a protocol π_{dsm} for distributed state management that realizes the functionality. As stated before, we intend for our protocol to be run on (multiple) SCPs, which we assume to be incorruptible. The functionality, and therefore the protocol, allows multiple groups of these SCPs, which should be able to notify each other and send key permissions to each other. In the protocol, this is done over some network. Since we stated that we assume insecure channels, but we require authenticated, confidential communication to realize the functionality, the protocol employs an AEAD scheme, which uses a key for each which we call communication key for that group. In our protocol, we assume that these communication keys are already distributed among all group members and we do not concern ourselves with how that is done in practice. We formalize this assumption into a functionality $\mathcal{F}_{\text{gsetup}}$ by taking inspiration from [11], where they define a similar functionality for groups of two people. This functionality is parametrized by

an algorithm KeyGen used to generate keys, which in π_{dsm} will be the key generation algorithm of the AEAD scheme. Then, parties can simply query $\mathcal{F}_{\text{gsetup}}$ to get their communication keys for a group when they need them.

Group Setup Functionality $\mathcal{F}_{\text{gsetup}}[\mathsf{KeyGen}]$

- Memory is per sid.

Request On input $(\texttt{key}, \mathsf{sid}, \mathcal{H})$ for some party $\mathcal{P} \in \mathcal{H}$

1: If there is a $(\texttt{key}, \mathsf{sid}, \mathcal{H}, k')$ in memory for some k', set $k \leftarrow k'$.
2: Else, generate $k \xleftarrow{\$} \mathsf{KeyGen}$ and store $(\texttt{key}, \mathsf{sid}, \mathcal{H}, k)$.
3: Send $(\texttt{key}, \mathsf{sid}, \mathcal{H}, k)$ to $\mathcal{P}$.

In practice, one could implement this functionality by either using a key derivation function to derive group communication keys from a shared master secret key, or by hard-coding all communication keys into each SCP.

The π_{dsm} protocol then closely follows the structure of $\mathcal{F}_{\text{dsm}}$, in that it provides the same interfaces, however now interfaces are defined for each party instead of an ideal functionality. Still, the interfaces roughly do similar things, as the protocol as a whole needs to somehow match or realize $\mathcal{F}_{\text{dsm}}$. Only the **Init** interface is a big exception, as it does nothing in π_{dsm}. Another important difference, as mentioned above, is that the design of π_{dsm} needs to consider the insecure network, for which we use the AEAD scheme to secure it. While the AEAD scheme guarantees confidentiality and authenticity, it does not prevent replay attacks, which is required to uphold the (property). To solve that the π_{dsm} uses counters. We define two sets of counters, one for sending and one for receiving messages. Both are parametrized by which group $\mathcal{H}$ the message is sent in, which party $\mathcal{P}_j$ is the sender of the message, and which party $\mathcal{P}_t$ is the receiver. We denote them by $\mathcal{C}_s^{\mathcal{H}}(j,t)$ for the sending counters and $\mathcal{C}_r^{\mathcal{H}}(j,t)$ for the receiving counters. Whenever a message is sent, the current $\mathcal{C}_s^{\mathcal{H}}(j,t)$ is put into the header of the AEAD scheme when encrypting the message, and afterwards the counter is increased. Whenever a message is received, after confirming the authenticity of the message by decrypting the ciphertext, the counter ctr in the header of the message is compared against $\mathcal{C}_r^{\mathcal{H}}(j,t)$. If $\mathcal{C}_r < \mathsf{ctr}$, the message is considered to be *fresh*, $\mathcal{C}_s^{\mathcal{H}}(j,t)$ is set to ctr and then the message is processed. Else, the message is discarded. Thus, it may happen that genuine messages are discarded, e.g. if messages arrive in a wrong order. Although this property is stronger than required by $\mathcal{F}_{\text{dsm}}$, it is a practical solution as only two counters per communication partner have to be stored. We refer to Section 6 for a discussion about alternatives.

We continue with describing the interfaces of the π_{dsm} protocol. In the **Key Generation** interface, a party $\mathcal{P}_j$ generates SwS keys using the KeyGen algorithm of an SwS scheme, stores it, and assigns all subkeys to itself by setting $\mathcal{B}_j^{\mathsf{pk}} \leftarrow [\ell_{\text{sub}}]$.

In the **Notify** interface an (already notified) party first retrieves its communication key for the desired group using $\mathcal{F}_{\text{gsetup}}$. Then, it creates a ciphertext by encrypting a header and a message, where the former contains the public key of the SwS scheme (as it is public information anyway) and the current counter $\mathcal{C}_s^{\mathcal{H}}(j,t)$, and the latter contains *all* subkeys. The party then increments $\mathcal{C}_s^{\mathcal{H}}(j,t)$ and sends the ciphertext over the (insecure) network to the party it wants to notify. In the protocol description, this is formalized by giving the ciphertext to the adversary $\mathcal{A}$, as he has control over the network. Due to this, the message does not immediately arrive at the target party.

The **Notified** is called by $\mathcal{A}$ or the network, when the notifying message arrives at the target party. The target party then simply decrypts the ciphertext and confirms the authenticity through that. It then checks the *freshness* of the received message by $\mathcal{C}_r^{\mathcal{H}}(j,t)$. If the message is fresh, it stores the key and updates $\mathcal{C}_s^{\mathcal{H}}(j,t)$. Note here that we do not need to store any flags $\mathcal{N}_j^{\mathsf{pk}}$. Instead, having a public key stored implies being notified about it.

When the **Assign** interface of a party is called, it first checks whether the subkeys req it needs to delegate to another party are currently assigned to itself. If that is the case, it unassigns them from itself, retrieves its communication key through $\mathcal{F}_{\text{gsetup}}$ and creates a ciphertext with req and $\mathcal{C}_s^{\mathcal{H}}(j,t)$ as the header. The message is set as the empty message symbol $\top$, as there is no secret information since req can be public. Then, $\mathcal{C}_s^{\mathcal{H}}(j,t)$ is increased and the ciphertext is sent to the network or $\mathcal{A}$.

When $\mathcal{A}$ calls the **Assigned** interface to indicate that a message transferring keys has arrived, the receiving party $\mathcal{P}_t$ checks the authenticity by decrypting the ciphertext to the empty message $\top$. Then, the freshness of the message is checked by comparing the counter from the header against $\mathcal{C}_r^{\mathcal{H}}(j,t)$. If the message is fresh, req is added to $\mathcal{B}_t^{\mathsf{pk}}$, and $\mathcal{C}_r^{\mathcal{H}}(j,t)$ is updated.

In the **Sign** interface, the specified subkey l is unassigned from party $\mathcal{P}_j$ by removing it from $\mathcal{B}_j^{\mathsf{pk}}$. Then, $\mathcal{P}_j$ uses the Sign algorithm of the SwS scheme to create the signature.

Similarly, in the **Verify** interface, we simply call the Vrfy algorithm of the SwS scheme.

We now define the distributed state management protocol π_{dsm}. For that, let λ be a fixed security parameter. Let $\Sigma = (\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Vrfy})$ be a signature scheme with subkeys. Let $\Pi = (\Pi.\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec})$ be an AEAD scheme. Define $\mathcal{F}_{\text{gsetup}} := \mathcal{F}_{\text{gsetup}}[\Pi.\mathsf{KeyGen}(1^\lambda)]$ for readability. We define the protocol π_{dsm} as follows:

Protocol π_{dsm}

- Each party ignores any message from any party that contains some session ID sid until after they have received $(\texttt{init}, \mathsf{sid})$.

- If a set is accessed although it has not been initialized, it is first initialized as $\emptyset$.
- When a counter $\mathcal{C}_r^{\mathsf{pk}}(j,t)$ or $\mathcal{C}_s^{\mathsf{pk}}(j,t)$ is accessed for the first time, it is first initialized to 0.
- Memory is per party and sid.

Initialization $\mathcal{P}_j$ on input $(\texttt{init}, \mathsf{sid})$, do nothing.

Key Generation $\mathcal{P}_j$ on input $(\texttt{keygen}, \mathsf{sid}, \mathcal{H})$

1: If $\mathcal{P}_j \notin \mathcal{H}$, output $\perp$.
2: Compute $((\mathsf{sk}_i)_{i\in[\ell_{\text{sub}}]}, \mathsf{pk}) \xleftarrow{\$} \mathsf{KeyGen}(1^\lambda)$.
3: Set $\mathcal{B}_j^{\mathsf{pk}} \leftarrow [\ell_{\text{sub}}]$.
4: Store $(\texttt{key}, \mathsf{sid}, \mathcal{H}, (\mathsf{sk}_i)_{i\in[\ell_{\text{sub}}]}, \mathsf{pk})$.
5: Output $(\texttt{pk}, \mathsf{sid}, \mathsf{pk}, \ell_{\text{sub}})$.

Notify $\mathcal{P}_j$ on input $(\texttt{notify}, \mathsf{sid}, \mathsf{pk}, \mathcal{P}_t)$

6: If $(\texttt{key}, \mathsf{sid}, \mathcal{H}, (\mathsf{sk}_i)_{i\in[\ell_{\text{sub}}^{\mathsf{pk}}]}, \mathsf{pk})$ is in memory for some $\mathcal{H}$, continue, else output $\perp$.
7: If $\mathcal{P}_j \notin \mathcal{H}$ or $\mathcal{P}_t \notin \mathcal{H}$, output $\perp$.
8: Set $h \leftarrow (\texttt{key}, \mathsf{pk})$ and $\mu = (\mathsf{sk}_i)_{i\in[\ell_{\text{sub}}]}$.
9: Send $(\texttt{key}, \mathsf{sid}, \mathcal{H})$ to $\mathcal{F}_{\text{gsetup}}$ and wait for answer $(\texttt{key}, \mathsf{sid}, \mathcal{H}, k)$.
10: Set $\mathsf{ctr} \leftarrow \mathcal{C}_s^{\mathcal{H}}(j,t)$.
11: Compute $c \xleftarrow{\$} \mathsf{Enc}(k, (\mathcal{H}, \mathcal{P}_j, \mathcal{P}_t, \mathsf{ctr}, h), \mu)$.
12: Set $\mathcal{C}_s^{\mathcal{H}}(j,t) \leftarrow \mathsf{ctr}+1$.
13: Send $(\texttt{send}, \mathsf{sid}, \mathcal{P}_j, \mathcal{P}_t, \mathsf{ctr}, (h,c))$ to $\mathcal{A}$.

Notified $\mathcal{P}_t$ on input $(\texttt{dlvr}, \mathsf{sid}, \mathcal{P}_j, \mathcal{P}_t, \mathsf{ctr}, (h,c))$ with $h = (\texttt{key}, \mathsf{pk})$ from $\mathcal{A}$

14: Send $(\texttt{key}, \mathsf{sid}, \mathcal{H})$ to $\mathcal{F}_{\text{gsetup}}$ and wait for answer $(\texttt{key}, \mathsf{sid}, \mathcal{H}, k)$.
15: Compute $\mu \xleftarrow{\$} \mathsf{Dec}(k, (\mathcal{H}, \mathcal{P}_j, \mathcal{P}_t, \mathsf{ctr}, h), c)$.
16: Parse $(\mathsf{sk}_i)_{i\in[\ell_{\text{sub}}]} \leftarrow \mu$.
17: If $\mathcal{C}_r^{\mathcal{H}}(j,t) < \mathsf{ctr}$, set $\mathcal{C}_r^{\mathcal{H}}(j,t) \leftarrow \mathsf{ctr}$, and store $(\texttt{key}, \mathsf{sid}, \mathcal{H}, (\mathsf{sk}_i)_{i\in[\ell_{\text{sub}}]}, \mathsf{pk})$, and set $\mathcal{B}_t^{\mathsf{pk}} \leftarrow \emptyset$.

Assign $\mathcal{P}_j$ on input $(\texttt{assign}, \mathsf{sid}, \mathsf{pk}, \mathcal{P}_t, \mathsf{req})$

18: If $(\texttt{key}, \mathsf{sid}, \mathcal{H}, \cdot, \mathsf{pk})$ exists in memory for some $\mathcal{H}$, continue, else output $\perp$.
19: If $\mathcal{P}_t \notin \mathcal{H}$, output $\perp$.
20: If $\mathsf{req} \not\subseteq \mathcal{B}_j^{\mathsf{pk}}$, output $\perp$.
21: Set $\mathcal{B}_j^{\mathsf{pk}} \xleftarrow{\setminus} \mathsf{req}$.
22: Set $h \leftarrow (\texttt{assgnd}, \mathsf{pk}, \mathsf{req})$ and $\mu \leftarrow \top$.
23: Send $(\texttt{key}, \mathsf{sid}, \mathcal{H})$ to $\mathcal{F}_{\text{gsetup}}$ and wait for answer $(\texttt{key}, \mathsf{sid}, \mathcal{H}, k)$.
24: Set $\mathsf{ctr} \leftarrow \mathcal{C}_s^{\mathcal{H}}(j,t)$.
25: Compute $c \xleftarrow{\$} \mathsf{Enc}(k, (\mathcal{H}, \mathcal{P}_j, \mathcal{P}_t, \mathsf{ctr}, h), \mu)$.
26: Set $\mathcal{C}_s^{\mathcal{H}}(j,t) \leftarrow \mathsf{ctr}+1$.
27: Send $(\texttt{send}, \mathsf{sid}, \mathcal{P}_j, \mathcal{P}_t, \mathsf{ctr}, (h,\mu))$ to $\mathcal{A}$.

Assigned On input $(\texttt{dlvr}, \mathsf{sid}, \mathcal{P}_j, \mathcal{P}_t, \mathsf{ctr}, (h,\mu))$ with $h = (\texttt{assgnd}, \mathsf{pk}, \mathsf{req})$ for party $\mathcal{P}_t$ from $\mathcal{A}$

28: If $(\texttt{key}, \mathsf{sid}, \mathcal{H}, \cdot, \mathsf{pk})$ is in memory for some $\mathcal{H}$, continue, else output $\perp$.
29: Send $(\texttt{key}, \mathsf{sid}, \mathcal{H})$ to $\mathcal{F}_{\text{gsetup}}$ and wait for answer $(\texttt{key}, \mathsf{sid}, \mathcal{H}, k)$.
30: Compute $\top \xleftarrow{\$} \mathsf{Dec}(k, (\mathcal{H}, \mathcal{P}_j, \mathcal{P}_t, \mathsf{ctr}, h), c)$.
31: If $\mathcal{C}_r^{\mathcal{H}}(j,t) < \mathsf{ctr}$, set $\mathcal{C}_r^{\mathcal{H}}(j,t) \leftarrow \mathsf{ctr}$ and set $\mathcal{B}_t^{\mathsf{pk}} \xleftarrow{\cup} \mathsf{req}$.
32: Output $(\texttt{keyset}, \mathsf{sid}, \mathsf{pk}, \mathcal{B}_t^{\mathsf{pk}})$.

Sign $\mathcal{P}_j$ on input $(\texttt{sign}, \mathsf{sid}, \mathsf{pk}, \mu, l)$

33: If $(\texttt{key}, \mathsf{sid}, \cdot, (\mathsf{sk}_i)_{i\in[\ell_{\text{sub}}]}, \mathsf{pk})$ is not in memory, output $\perp$.
34: If $l \notin \mathcal{B}_j^{\mathsf{pk}}$, output $\perp$.
35: Set $\mathcal{B}_j^{\mathsf{pk}} \xleftarrow{\setminus} \{l\}$.
36: Compute $\sigma \xleftarrow{\$} \mathsf{Sign}(1^\lambda, \mathsf{sk}_l, \mu)$ and output σ.

Verify $\mathcal{P}$ on input $(\texttt{vrfy}, \mathsf{sid}, \mathsf{pk}, \mu, \sigma)$

37: Compute $b \xleftarrow{\$} \mathsf{Vrfy}(1^\lambda, \mathsf{pk}, \mu, \sigma)$.
38: Output $(\texttt{verified}, \mathsf{sid}, \mathsf{pk}, \mu, \sigma, b)$.

Note that the inputs for the **Notified** and **Assigned** interfaces are slightly different than in $\mathcal{F}_{\text{dsm}}$. This is because $\mathcal{F}_{\text{dsm}}$ does not need to communicate over the network while π_{dsm} needs to use it.

Theorem 5.1 (Informal)**.** *Protocol π_{dsm} realizes functionality $\mathcal{F}_{\text{dsm}}$ if the signature scheme with subkeys Σ is strong EUF-CMA one-time secure (Definition 2.2) and linear-time (Definition 2.3) and if the AEAD scheme Π is CPA secure (Definition 2.5) and unforgeable (Definition 2.6).*

Proof outline. The proof is divided into two parts. We first encapsulate the communication of protocol π_{dsm} into a new functionality $\mathcal{F}_{\text{fresh}}$. Intuitively, this functionality guarantees the unforgeability and confidentiality of the messages sent to the attacker $\mathcal{A}$ and prevents $\mathcal{A}$ from delivering the same message multiple times to the receiver. In Corollary A.3 we show that this functionality is realizable by some protocol in the $\mathcal{F}_{\text{gsetup}}$-hybrid model. In fact, this protocol consists of the code related to communication from protocol π_{dsm}, i.e. the code used to compute header h, message μ, or counter ctr and to communicate with $\mathcal{A}$. It uses the AEAD scheme Π to ensure confidentiality and unforgeability and the pairwise counters $\mathcal{C}_s^{\mathcal{H}}(j,t), \mathcal{C}_r^{\mathcal{H}}(j,t)$ to guarantee that messages are never delivered multiple times.

After the communication has been outsourced to the realizable functionality $\mathcal{F}_{\text{fresh}}$, we can replace the corresponding lines of the protocol π_{dsm} by calls to $\mathcal{F}_{\text{fresh}}$. This results in a protocol, $\pi_{\text{f-dsm}}$, in the $\mathcal{F}_{\text{fresh}}$-hybrid model, which by using the ideal communication provided by $\mathcal{F}_{\text{fresh}}$ directly upholds the *subkey partition* property, namely that key permissions are assigned to exactly one party, or in transit, or spent. Furthermore, outsourcing communication to $\mathcal{F}_{\text{fresh}}$ ensures the attacker i) cannot view the subkeys sent in **Notify/Notified** and ii) cannot change or send permissions multiple times when transferring permissions in **Assign/Assigned**. Hence, the signature scheme's strong EUF-CMA security is preserved throughout the execution.

We proceed by showing that $\pi_{\text{f-dsm}}$ realizes $\mathcal{F}_{\text{dsm}}$ in the $\mathcal{F}_{\text{fresh}}$-hybrid model. Our approach is similar to the proof of Lemma 3.5 from [5]. In particular, we align the ideal experiment and the real experiment by stepwise transferring code from $\mathcal{F}_{\text{dsm}}$ into the protocol $\pi_{\text{f-dsm}}$. For each hybrid we prove that no efficient environment can distinguish it from the previous one. We use, for example, that SwS Σ is deterministic (cf. Definition 2.1), is strong EUF-CMA secure (Def-

inition 2.2), and is linear-time (Definition 2.3). The latter reductions only work, because we show by an invariant that protocol $\pi_{\text{f-dsm}}$ upholds the *subkey partition* property and thus never uses subkeys more than once for signing.

Finally, we end up with a hybrid experiment, which is almost identical to the ideal experiment. The only difference being that parties in the hybrid experiment transfer knowledge and permissions of subkeys directly via $\mathcal{F}_{\text{fresh}}$ while in the ideal world experiment there is no such inter-party communication. We construct a simulator $\mathcal{S}$ which bridges this last gap between real world and ideal world. It is based on the fact that in $\mathcal{F}_{\text{dsm}}$ the **Notify/Notified** and **Assign/Assigned** are used to transfer knowledge and permissions of subkeys. The $\mathcal{S}$ accomplishes this by internally simulating a copy of $\mathcal{F}_{\text{fresh}}$.

By applying the UC composition theorem, i.e., that the calls to $\mathcal{F}_{\text{fresh}}$ in $\pi_{\text{f-dsm}}$ can securely be replaced by the protocol realizing $\mathcal{F}_{\text{fresh}}$, we obtain protocol π_{dsm} and Theorem 5.1 follows. The full proof of this theorem is given in Section B.

□

6. Extensions and Future Work

In this section, we outline possible extensions of our model and directions for future research.

Weaker Assumptions. Our current treatment of signature schemes with subkeys (SwS) relies on two simplifying assumptions that facilitate the proof of Theorem 5.1. Concretely, our Definition 2.1 requires every SwS to satisfy *perfect* correctness and *deterministic* verification. To the best of our knowledge, these assumptions hold for all practical signature schemes with subkeys that fit our syntax, so they do not limit real-world applicability. Nevertheless, the security proof can be adapted to weaker notions. Replacing perfect correctness by *computational* correctness merely changes all perfect indistinguishability arguments into computational ones. If we further drop deterministic verification, the protocol still remains UC-secure, provided we assume consistency of verification queries, as defined and proven in [5]. Intuitively, the consistency property guarantees that, for any (even maliciously generated) public key pk, verification of a message-signature pair (μ, σ) yields the same result except with negligible probability. Ensuring consistency with respect to adversarially generated public keys is essential, since every public key submitted to the protocol interfaces is recorded in the set $\mathcal{K}_{\text{pk}}$ during the ideal-world execution.

Weaker Communication Channels. We may also employ a weaker functionality for communication than $\mathcal{F}_{\text{fresh}}$. We chose $\mathcal{F}_{\text{fresh}}$ because it keeps the realization clean and simple. For each pair of parties $(\mathcal{P}_j, \mathcal{P}_t)$, only two counters keeping track of messages sent and received must be stored. This design, however, has the disadvantage that if messages are reordered by the network, whether adversarially or accidentally, then intermediate messages are lost. In our protocol this means that the corresponding subkeys are lost and will not be used for signing anymore.

One possible remedy, which still realizes $\mathcal{F}_{\text{fresh}}$, is to cache messages and process them at appropriate times. For example, messages whose counters lie far ahead of the last received counter can be stored temporarily, and are only processed if the the difference to the counter of the latest accepted message is small enough, or if a certain time span has passed.

Another, potentially preferable, option is to use a new weaker functionality. For communication, our proof effectively relies on the properties that messages cannot be modified, that encryption is CPA-secure, and that each message is delivered at most once. The latter property can be achieved by having each party store the set of *all* counters it has received from $\mathcal{P}_j$, rather than keeping only the most recent, i.e. highest, counter. When a message arrives, if its counter is already contained in the set, the message is discarded. If a message with a new counter arrives, it is accepted and the counter is added to the set. This approach is particularly advisable when the participating devices possess sufficient, incorruptible local storage, which they need to store the subkey permissions anyway. However, this solution entails a bigger storage requirement on this special storage, which is why we chose to use the method presented in $\mathcal{F}_{\text{fresh}}$ and π_{fresh}.

Distributed Key Generation. Currently, neither our functionality nor our protocol explicitly support distributed key generation in a natural way. As future work, one may explore different variants of distributed key generation within our framework, as illustrated by the three examples discussed below.

One approach is to let each participating party instantiate our protocol π_{dsm} and generate a public key pk. The parties then combine their individual public keys using a method such as Merkle trees (cf. Figure 1) to derive a joint public key. In this setup, every party begins with its own set of subkeys that can be used for signing, while still needing to record which subkeys have already been consumed. Then, if some party exhausts its available subkeys, it could obtain additional ones from other parties. Bookkeeping, as well as the transfer of subkeys and their corresponding permissions, is already handled by our protocol. Thus, this approach merely requires appropriately modeling the process for combining public keys in a UC functionality and realizing it with our functionality in the $\mathcal{F}_{\text{dsm}}$-hybrid model.

Another solution that employs a shared public key but keeps its secret subkeys always local can also be appealing in scenarios where transferring (sub)keys is legally restricted or technically infeasible. In such cases, neither the functionality for distributed state management nor the protocol must offer an interface such as **Notify** for subkey exchange.

Finally, one may employ UC-secure distributed key-generation protocols instead of letting a single party execute the KeyGen algorithm. Such an approach would yield an initial distribution of secret subkeys among the participating parties but would likely incur significant computational and communi-

cation overhead.

While the first approach is comparatively easy to put into practice and theory, it should be noted that the latter variants require more modifications and introduce numerous subtleties to both our main functionality and its realization. In particular, adapting the consistency guarantees and interface structure to support distributed key generation would necessitate a careful re-design of functionality and security arguments. We therefore leave a detailed investigation of these variants to future work.

Adapting Notify. In some scenarios, transferring secret (sub)keys may be legally permitted only through special channels. Instead of removing the **Notify** interface entirely, as suggested in the previous paragraph, one could therefore consider variants that adapt it to such constrained communication settings. For example, **Notify** could rely on a functionality that models the physical transfer of subkeys, where keys from one SCP are cloned to another SCP via a specialized process that an adversary cannot interfere with. Since this operation would be costly in practice, it should be used only rarely. Whenever subkey transfer via usual (secured) channels is permitted, our original protocol remains the more efficient option.

Because replaying messages poses no problem when notifying, one could also employ a weaker communication functionality that does not prevent replay attacks in contrast to $\mathcal{F}_{\text{fresh}}$. This relaxation could lead to even more efficient protocol instantiations.

Another possible direction is to remove the explicit **Notify** interface altogether, yielding a functionality that appears more natural for distributed key management. If a party holds permission for a key sk_i that should be assigned to another party $\mathcal{P}_t$, it might seem cumbersome to first send all subkeys via **Notify** rather than invoking **Assign** directly. However, due to our choices in formalizing SwS schemes (cf. Section 2.2) this simplification either requires some form of consensus on key ownership or forces the encrypted retransmission of all subkeys each time an assignment occurs, which entails higher communication costs. To reduce this cost, it is possible, for example, to use an SwS scheme where the vector of subkeys can be represented by a single seed for a pseudo-random generator. Thus, only the seed needs to be encrypted instead of all subkeys. Adapting our protocol and functionality to this scenario then only requires minor changes.

References

[1] Ross J. Anderson et al. “Cryptographic Processors-A Survey”. In: *Proc. IEEE* 94.2 (2006), pp. 357–369. DOI: 10.1109/JPROC.2005.862423. URL: https://doi.org/10.1109/JPROC.2005.862423.

[2] Johannes Buchmann, Erik Dahmen, and Andreas Hülsing. “XMSS - A Practical Forward Secure Signature Scheme Based on Minimal Security Assumptions”. In: *Post-Quantum Cryptography - 4th International Workshop, PQCrypto 2011, Taipei, Taiwan, November 29 - December 2, 2011. Proceedings*. Ed. by Bo-Yin Yang. Vol. 7071. Lecture Notes in Computer Science. Springer, 2011, pp. 117–129. DOI: 10.1007/978-3-642-25405-5. URL: https://doi.org/10.1007/978-3-642-25405-5.

[3] Ran Canetti. “Security and Composition of Cryptographic Protocols: A Tutorial”. In: *Secure Multi-Party Computation*. Ed. by Manoj Prabhakaran and Amit Sahai. Vol. 10. Cryptology and Information Security Series. IOS Press, 2013, pp. 61–119. DOI: 10.3233/978-1-61499-169-4-61. URL: https://doi.org/10.3233/978-1-61499-169-4-61.

[4] Ran Canetti. “Universally Composable Security”. In: *J. ACM* 67.5 (2020), 28:1–28:94. DOI: 10.1145/3402457. URL: https://doi.org/10.1145/3402457.

[5] Ran Cohen et al. “An Unstoppable Ideal Functionality for Signatures and a Modular Analysis of the Dolev-Strong Broadcast”. In: *IACR Cryptol. ePrint Arch.* (2024). URL: https://eprint.iacr.org/2024/1807.

[6] David A Cooper et al. “Recommendation for stateful hash-based signature schemes”. In: *NIST Special Publication* 800.208 (2020), pp. 800–208.

[7] C. Dods, Nigel P. Smart, and Martijn Stam. “Hash Based Digital Signature Schemes”. In: *Cryptography and Coding, 10th IMA International Conference, Cirencester, UK, December 19-21, 2005, Proceedings*. Ed. by Nigel P. Smart. Vol. 3796. Lecture Notes in Computer Science. Springer, 2005, pp. 96–115. DOI: 10.1007/11586821. URL: https://doi.org/10.1007/11586821.

[8] Oded Goldreich. *The Foundations of Cryptography - Volume 1: Basic Techniques*. Cambridge University Press, 2001. ISBN: 0-521-79172-3. DOI: 10.1017/CBO9780511546891. URL: http://www.wisdom.weizmann.ac.il/~oded/foc-vol1.html.

[9] Oded Goldreich. *The Foundations of Cryptography - Volume 2: Basic Applications*. Cambridge University Press, 2004. ISBN: 0-521-83084-2. DOI: 10.1017/CBO9780511721656. URL: http://www.wisdom.weizmann.ac.il/~oded/foc-vol2.html.

[10] Dennis Hofheinz and Victor Shoup. “GNUC: A New Universal Composability Framework”. In: *J. Cryptol.* 28.3 (2015), pp. 423–508. DOI: 10.1007/S00145-013-9160-Y. URL: https://doi.org/10.1007/s00145-013-9160-y.

[11] Ralf Küsters and Max Tuengerthal. “Universally Composable Symmetric Encryption”. In: *Proceedings of the 22nd IEEE Computer Security Foundations Symposium, CSF 2009, Port Jefferson, New York, USA, July 8-10, 2009*. IEEE Computer Society, 2009, pp. 293–307. DOI: 10.1109/CSF.2009.18. URL: https://doi.org/10.1109/CSF.2009.18.

[12] Leslie Lamport. *Constructing digital signatures from a one-way function*. SRI International Computer Science Laboratory. 1979.

[13] Frank T Leighton and Silvio Micali. *Method and system for personal identification using proofs of legitimacy*. US Patent 4,995,081. Feb. 1991.

[14] David A. McGrew, Michael Curcio, and Scott R. Fluhrer. "Leighton-Micali Hash-Based Signatures". In: *RFC* 8554 (2019), pp. 1–61. DOI: 10.17487/RFC8554. URL: https://doi.org/10.17487/RFC8554.

[15] David A. McGrew et al. "State Management for Hash-Based Signatures". In: *Security Standardisation Research - Third International Conference, SSR 2016, Gaithersburg, MD, USA, December 5-6, 2016, Proceedings*. Ed. by Lidong Chen, David A. McGrew, and Chris J. Mitchell. Vol. 10074. Lecture Notes in Computer Science. Springer, 2016, pp. 244–260. DOI: 10.1007/978-3-319-49100-4. URL: https://doi.org/10.1007/978-3-319-49100-4.

[16] Phillip Rogaway. "Authenticated-encryption with associated-data". In: *Proceedings of the 9th ACM Conference on Computer and Communications Security, CCS 2002, Washington, DC, USA, November 18-22, 2002*. Ed. by Vijayalakshmi Atluri. ACM, 2002, pp. 98–107. DOI: 10.1145/586110.586125. URL: https://doi.org/10.1145/586110.586125.

[17] Douglas Wikström. "Simplified Universal Composability Framework". In: *Theory of Cryptography - 13th International Conference, TCC 2016-A, Tel Aviv, Israel, January 10-13, 2016, Proceedings, Part I*. Ed. by Eyal Kushilevitz and Tal Malkin. Vol. 9562. Lecture Notes in Computer Science. Springer, 2016, pp. 566–595. DOI: 10.1007/978-3-662-49096-9. URL: https://doi.org/10.1007/978-3-662-49096-9.

Appendix A. Additional Functionalities

In the proof outline of Theorem 5.1 we explained that to prove the theorem, we introduce another functionality $\mathcal{F}_{\text{fresh}}$ and show that it is realizable in Corollary A.3. However, in the proof for the corollary we use yet another functionality, which we call the *secure group channel* functionality $\mathcal{F}_{\text{sgc}}$. We first define $\mathcal{F}_{\text{sgc}}$ and show that it is realizable in the $\mathcal{F}_{\text{gsetup}}$-hybrid world. Then, we define $\mathcal{F}_{\text{fresh}}$ and show that it is realizable in the $\mathcal{F}_{\text{sgc}}$-hybrid world, proving Corollary A.3.

A.1. Secure Group Channel

The secure group channel functionality allows authenticated, confidential communication between members of a group. However, in contrast to the properties of $\mathcal{F}_{\text{fresh}}$ mentioned in the proof of Theorem 5.1, it does *not* prevent messages from being duplicated. $\mathcal{F}_{\text{sgc}}$ has two interfaces, **Send** and **Deliver**. When a party calls **Send** with a communication group, target party, and a message with a header, $\mathcal{F}_{\text{sgc}}$ assigns a identifier to this message, where the identifier is unique to the communication group. It then stores this information in memory and give it to the simulator, except that it does not send the message to the simulator (since it should be confidential) but only the length of the message, which is the leakage. The simulator can then call the **Deliver** interface with a communication group, origin party, target party, header and an identifier to make $\mathcal{F}_{\text{sgc}}$ deliver send the message identified by the communication group and the identifier to the correct target. $\mathcal{F}_{\text{sgc}}$ only does so, if it remembers the message from the **Send** interface.

Secure Group Channel Functionality $\mathcal{F}_{\text{sgc}}$

- Memory is per sid.
- When $\mathsf{id}_{\mathcal{H}}$ is accessed for the first time, it is initialized to $\mathsf{id}_{\mathcal{H}} \leftarrow 0$.

Send On input $(\texttt{send}, \mathsf{sid}, \mathcal{H}, \mathcal{P}_t, h, \mu)$ from $\mathcal{P}_j$

1: If $\mathcal{P}_j \notin \mathcal{H}$ or $\mathcal{P}_t \notin \mathcal{H}$ output $\perp$.
2: Store $(\mathcal{H}, \mathcal{P}_j, \mathcal{P}_t, \mathsf{id}_{\mathcal{H}}, h, \mu)$.
3: Set $\mathsf{id}_{\mathcal{H}} \leftarrow \mathsf{id}_{\mathcal{H}} + 1$.
4: Send $(\texttt{send}, \mathsf{sid}, \mathcal{H}, \mathcal{P}_j, \mathcal{P}_t, \mathsf{id}_{\mathcal{H}}, h, |\mu|)$ to $\mathcal{S}$.

Deliver On input $(\texttt{dlvr}, \mathsf{sid}, \mathcal{H}, \mathcal{P}_j, \mathcal{P}_t, \mathsf{id})$ from $\mathcal{S}$

5: If $(\mathcal{H}, \mathcal{P}_j, \mathcal{P}_t, \mathsf{id}, h, \mu)$ exists in memory for some h, μ, send $(\texttt{dlvr}, \mathsf{sid}, \mathcal{H}, \mathcal{P}_j, h, \mu)$ to $\mathcal{P}_t$.
6: Else, send $\perp$ to $\mathcal{S}$.

We now construct a protocol π_{sgc} realizing $\mathcal{F}_{\text{sgc}}$. It uses an AEAD scheme to ensure authenticity and confidentiality and works in the $\mathcal{F}_{\text{gsetup}}$-hybrid model to get the communication keys for the AEAD scheme. For that, let λ be a fixed security parameter. Let $(\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec})$ be an AEAD scheme. Define $\mathcal{F}_{\text{gsetup}} := \mathcal{F}_{\text{gsetup}}[\mathsf{KeyGen}(1^\lambda)]$ for readability. We define the secure group channel protocol π_{sgc} as follows.

Secure Group Channel Protocol π_{sgc}

- Let $(\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec})$ be an AEAD scheme.
- Memory is per party and sid.

Send $\mathcal{P}_j$ on input $(\texttt{send}, \mathsf{sid}, \mathcal{H}, \mathcal{P}_t, h, \mu)$

1: If $\mathcal{P}_j \notin \mathcal{H}$ or $\mathcal{P}_t \notin \mathcal{H}$, output $\perp$.
2: Send $(\texttt{key}, \mathsf{sid}, \mathcal{H})$ to $\mathcal{F}_{\text{gsetup}}$ and wait for answer $(\texttt{key}, \mathsf{sid}, \mathcal{H}, k)$.
3: Compute $c \xleftarrow{\$} \mathsf{Enc}(k, (\mathcal{H}, \mathcal{P}_j, \mathcal{P}_t, h), \mu)$.
4: Set $c' \leftarrow ((\mathcal{H}, \mathcal{P}_j, \mathcal{P}_t, h), c)$.
5: Send $(\texttt{send}, \mathsf{sid}, \mathcal{H}, \mathcal{P}_j, \mathcal{P}_t, c')$ to $\mathcal{A}$.

Deliver $\mathcal{P}_t$ on input $(\texttt{dlvr}, \mathsf{sid}, \mathcal{H}, \mathcal{P}_j, \mathcal{P}_t, c')$ from $\mathcal{A}$

6: Send $(\texttt{key}, \mathsf{sid}, \mathcal{H})$ to $\mathcal{F}_{\text{gsetup}}$ and wait for answer $(\texttt{key}, \mathsf{sid}, \mathcal{H}, k)$.
7: Parse $((\mathcal{H}, \mathcal{P}_j, \mathcal{P}_t, h), c) \leftarrow c'$.
8: Compute $\mu' \xleftarrow{\$} \mathsf{Dec}(k, (\mathcal{H}, \mathcal{P}_j, \mathcal{P}_t, h), c)$.
9: If $\mu' \neq \perp$, send $(\texttt{dlvr}, \mathsf{sid}, \mathcal{H}, \mathcal{P}_j, h, \mu')$ to the environment.

10: Else, send $\perp$ to $\mathcal{A}$.

Theorem A.1. *Protocol π_{sgc} realizes functionality $\mathcal{F}_{\text{sgc}}$ in the $\mathcal{F}_{\text{gsetup}}$-hybrid model if the AEAD scheme is CPA secure (Definition 2.5) and unforgeable (Definition 2.6).*

Proof. To show this, we start with the real world of experiment with protocol π_{sgc} and adversary $\mathcal{A}$. Stepwise, we transform the real world experiment into the ideal world experiment, so that an environment cannot distinguish these steps. In each step we describe the additions or changes to the previous hybrid $\mathcal{H}_{\text{i}}$ and prove the resulting $\mathcal{H}_{\text{i+1}}$ computationally (or perfectly) indistinguishable from $\mathcal{H}_{\text{i}}$ for any efficient environment.

Hybrid $\mathcal{H}_0$: This hybrid is the real world experiment with protocol π_{sgc}, environment $\mathcal{E}$, and adversary $\mathcal{A}$.

Hybrid $\mathcal{H}_1$: In **Send**, after a party computes c' for some header h and message μ, it additionally stores $(\mathcal{H}, \mathcal{P}_j, \mathcal{P}_t, h, \mu, c')$ in global memory. In **Deliver**, a party first checks whether $(\mathcal{H}, \mathcal{P}_j, \mathcal{P}_t, h, \mu, c')$ exists in global memory for some h, μ. If it does not exist, it outputs $\perp$ to $\mathcal{A}$. Else, it continues executing **Deliver**.
The hybrids $\mathcal{H}_0$ and $\mathcal{H}_1$ are computationally indistinguishable due to the unforgeability of the AEAD scheme. An environment can only distinguish between $\mathcal{H}_0$ and $\mathcal{H}_1$ if the adversary submits an input to **Deliver** such that the decryption outputs some non-error message, while the ciphertext was not created by **Send**. Therefore, it is straightforward to construct an adversary with non-negligible winning probability against the unforgeability of the AEAD scheme, if there exists an environment that can distinguish between $\mathcal{H}_0$ and $\mathcal{H}_1$. The reduction simulates this environment and $\mathcal{H}_1$, except that it guesses for which public key the environment will produce an input to **Deliver** so that it can distinguish. For this public key, the reduction replaces the generated public key by its challenge public key and uses its encryption oracle to generate signatures when needed. If the environment is successful in distinguishing, the reduction knows a forgery for its challenge public key which was not created by the encryption oracle (since the input created by the environment was not created by **Send**), and thus the reduction wins the unforgeability game.

Hybrid $\mathcal{H}_2$: After a party computes μ' from some c' in **Deliver**, if $\mu' \neq \perp$ it sets $\mu' \leftarrow \mu$ instead where $(\mathcal{H}, \mathcal{P}_j, \mathcal{P}_t, h, \mu, c')$ is in global memory for some h, μ.
The only way the environment can distinguish between $\mathcal{H}_1$ and $\mathcal{H}_2$ is when the decryption of a ciphertext in the **Deliver** interface yields a different message than what was used to create the ciphertext and thus saved in memory in **Send**. Therefore, by the correctness of the AEAD scheme, we have $\mathcal{H}_1 \equiv \mathcal{H}_2$.

Hybrid $\mathcal{H}_3$: Parties encrypt $0^{|\mu|}$ instead of μ. By the CPA security of the AEAD, we immediately have that $\mathcal{H}_2$ is computationally indistinguishable from $\mathcal{H}_3$. Fix an sid and define an intermediate hybrid that is the same as $\mathcal{H}_2$, except that for sid parties encrypt $0^{|\mu|}$ instead of μ. If there was an environment $\mathcal{E}$ that could distinguish between $\mathcal{H}_2$ and the intermediate hybrid, we can construct a reduction that simulates $\mathcal{E}$ in $\mathcal{H}_2$, except that for the fixed sid it uses its encryption oracle to produce ciphertexts instead of using Enc. Then, $\mathcal{E}$ and the reduction have the same distinguishing advantage in their respective games. One can then repeat this for other intermediate hybrids, encrypting $0^{|\mu|}$ instead of μ for more and more fixed sid until one arrives at $\mathcal{H}_3$, where the message is replaced for every sid.

Hybrid $\mathcal{H}_4$: Whenever some $(\mathcal{H}, \mathcal{P}_j, \mathcal{P}_t, h, \mu, c')$ is stored in global memory (as defined in $\mathcal{H}_1$), instead $(\mathcal{H}, \mathcal{P}_j, \mathcal{P}_t, h, \mu), c', \mathsf{id}_{\mathcal{H}})$ is stored, where $\mathsf{id}_{\mathcal{H}}$ is a counter that starts at 0 and is increased by 1 after computing a ciphertext. Since we just store additional information, this is indistinguishable from $\mathcal{H}_3$.

We define the simulator $\mathcal{S}_{\text{sgc}}$.

- $\mathcal{S}_{\text{sgc}}$ simulates $\mathcal{H}_4$ internally. That means he simulates $\mathcal{A}$ and $\mathcal{F}_{\text{gsetup}}$ internally, forwarding any messages between $\mathcal{A}$ and the environment.
- When $\mathcal{S}_{\text{sgc}}$ receives $(\texttt{send}, \mathsf{sid}, \mathcal{H}, \mathcal{P}_j, \mathcal{P}_t, \mathsf{id}, h, \ell_m)$ as input, it simulates $\mathcal{P}_j$ as in $\mathcal{H}_4$ on input $(\texttt{send}, \mathsf{sid}, \mathcal{H}, \mathcal{P}_t, h, 0^{\ell_m})$.
- When $\mathcal{A}$ sends $(\texttt{dlvr}, \mathsf{sid}, \mathcal{H}, \mathcal{P}_j, \mathcal{P}_t, c')$ to some simulated $\mathcal{P}_t$, $\mathcal{S}_{\text{sgc}}$ simulates $\mathcal{P}_t$ in $\mathcal{H}_4$ on this input. If $\mathcal{P}_t$ then wants to send $(\texttt{dlvr}, \mathsf{sid}, \mathcal{H}, \mathcal{P}_j, h, \mu)$ to the environment, the simulator instead sends $(\texttt{dlvr}, \mathsf{sid}, \mathcal{H}, \mathcal{P}_j, \mathcal{P}_t, \mathsf{id})$ to $\mathcal{F}_{\text{sgc}}$, where $(\mathcal{H}, \mathcal{P}_j, \mathcal{P}_t, \mathsf{id}, h, \mu, \cdot)$ is in global memory for some id.

We now need to argue that no environment can distinguish between the simulator in the ideal world and $\mathcal{H}_4$. We start by arguing that the counters $\mathsf{id}_{\mathcal{H}}$ kept by $\mathcal{F}_{\text{sgc}}$ and by $\mathcal{S}_{\text{sgc}}$ added in $\mathcal{H}_4$ are the same. Whenever $\mathcal{F}_{\text{sgc}}$ increases a counter, it sends $(\texttt{send}, \mathsf{sid}, \mathcal{H}, \mathcal{P}_j, \mathcal{P}_t, \mathsf{id}_{\mathcal{H}}, h, |\mu|)$ to $\mathcal{S}_{\text{sgc}}$, who then simulates $\mathcal{P}_j$ internally, where the corresponding $\mathsf{id}_{\mathcal{H}}$ is increased as well. With a similar argument we can show that the $(\mathcal{H}, \mathcal{P}_j, \mathcal{P}_t, \mathsf{id}_{\mathcal{H}}, \mu, c')$ matches the $(\mathcal{H}, \mathcal{P}_j, \mathcal{P}_t, \mathsf{id}_{\mathcal{H}}, \mu)$ which is stored by the functionality, i.e. the simulator does correct bookkeeping. Therefore, whenever a party sends $(\texttt{dlvr}, \mathsf{sid}, \mathcal{H}, \mathcal{P}_j, h, \mu')$ to the environment in the hybrid world, the party simulated by $\mathcal{S}_{\text{sgc}}$ wants to do the same and $\mathcal{S}_{\text{sgc}}$ instead sends $(\texttt{dlvr}, \mathsf{sid}, \mathcal{H}, \mathcal{P}_j, \mathcal{P}_t, \mathsf{id})$ with the correct id to the functionality. Since the bookkeeping by the simulator was correct, $\mathcal{F}_{\text{sgc}}$ then sends $(\texttt{dlvr}, \mathsf{sid}, \mathcal{H}, \mathcal{P}_j, h, h, \mu')$ to the party which forwards it to the environment. Thus, in both the ideal and hybrid world, the environment gets the same input from the parties when **Deliver** is used. Furthermore,

the same holds trivially for **Send** since $\mathcal{S}_{\text{sgc}}$ simulates the protocol perfectly internally, and the messages of the simulated $\mathcal{A}$ to and from the environment are forwarded by $\mathcal{S}_{\text{sgc}}$, thus we have that

$$\{\text{HYBRID}^{\mathcal{F}_{\text{gsetup}}}_{\pi_{\text{sgc}},\mathcal{A},\mathcal{E}}(\kappa,z)\}_{\kappa\in\mathbb{N},z\in\{0,1\}^*} \approx_{\text{c}} \{\text{IDEAL}_{\mathcal{F}_{\text{sgc}},\mathcal{S}_{\text{sgc}},\mathcal{E}}(\kappa,z)\}_{\kappa\in\mathbb{N},z\in\{0,1\}^*}.$$

□

A.2. Fresh Functionality

Having defined $\mathcal{F}_{\text{sgc}}$, we can now define $\mathcal{F}_{\text{fresh}}$. In addition to the guarantees of $\mathcal{F}_{\text{sgc}}$, $\mathcal{F}_{\text{fresh}}$ should also prevent the ideal adversary from duplicating messages. This is done by utilizing counters $\mathcal{C}^{\mathcal{H}}_s(j,t), \mathcal{C}^{\mathcal{H}}_r(j,t)$ as described in Section 4.

Fresh Functionality $\mathcal{F}_{\text{fresh}}$

- Memory is per sid.
- When a counter $\mathcal{C}^{\mathsf{pk}}_r(j,t)$ or $\mathcal{C}^{\mathsf{pk}}_s(j,t)$ is accessed for the first time, it is first initialized to 0.

Send On input $(\texttt{send}, \mathsf{sid}, \mathcal{H}, \mathcal{P}_t, h, \mu)$ from $\mathcal{P}_j$

1: If $\mathcal{P}_j \notin \mathcal{H}$ or $\mathcal{P}_t \notin \mathcal{H}$, output $\perp$.
2: Set $\mathsf{ctr} \leftarrow \mathcal{C}^{\mathcal{H}}_s(j,t)$.
3: Store $(\mathcal{H}, \mathcal{P}_j, \mathcal{P}_t, \mathsf{ctr}, h, \mu)$.
4: Set $\mathcal{C}^{\mathcal{H}}_s(j,t) \leftarrow \mathsf{ctr}+1$.
5: Send $(\texttt{send}, \mathsf{sid}, \mathcal{H}, \mathcal{P}_j, \mathcal{P}_t, \mathsf{ctr}, h, |\mu|)$ to $\mathcal{S}$.

Deliver On input $(\texttt{dlvr}, \mathsf{sid}, \mathcal{H}, \mathcal{P}_j, \mathcal{P}_t, \mathsf{ctr})$ from $\mathcal{S}$

6: If $(\mathcal{H}, \mathcal{P}_j, \mathcal{P}_t, \mathsf{ctr}, h, \mu)$ is in memory for some h, μ and if $\mathcal{C}^{\mathcal{H}}_r(j,t) < \mathsf{ctr}$, set $\mathcal{C}^{\mathcal{H}}_r(j,t) \leftarrow \mathsf{ctr}$ and send $(\texttt{dlvr}, \mathsf{sid}, \mathcal{H}, \mathcal{P}_j, h, \mu)$ to $\mathcal{P}_t$.
7: Else, send $\perp$ to $\mathcal{S}$.

To construct a protocol realizing $\mathcal{F}_{\text{fresh}}$, we utilize $\mathcal{F}_{\text{sgc}}$ as it guarantees a subset of the requirements stated by $\mathcal{F}_{\text{fresh}}$ and therefore define a protocol π_{fresh} in the $\mathcal{F}_{\text{sgc}}$-hybrid model.

Theorem A.2. *There exists a protocol π_{fresh} that realizes $\mathcal{F}_{\text{fresh}}$ in the $\mathcal{F}_{\text{sgc}}$-hybrid model.*

Proof. In the functionality $\mathcal{F}_{\text{fresh}}$, each party $\mathcal{P}_i$ has an outgoing message counter $\mathcal{C}^{\mathcal{H}}_s(i,j)$ and an incoming message counter $\mathcal{C}^{\mathcal{H}}_r(j,i)$ for each other party $\mathcal{P}_j$. Since each counter does not interact with other counters and can be considered local to each party, in the protocol π_{fresh} each party can keep their own counters and update them accordingly when sending or receiving messages. Thus, a party initializes their own counters when they are first used, the same way as in $\mathcal{F}_{\text{fresh}}$. To send messages, the parties use the `send` interface of $\mathcal{F}_{\text{sgc}}$. It is then straightforward to show there exists a simulator $\mathcal{S}_{\text{fresh}}$ such that the environment cannot distinguish between $\mathcal{F}_{\text{fresh}}$ with $\mathcal{S}_{\text{fresh}}$ and π_{fresh} with some $\mathcal{A}$. $\mathcal{S}_{\text{fresh}}$ internally simulates π_{fresh}, $\mathcal{F}_{\text{sgc}}$ and $\mathcal{A}$, and then forwards any messages between them, themselves and the environment. Whenever a simulated party would compute a ciphertext using some μ, it instead uses $|\mu|$, since only the latter is known to $\mathcal{S}_{\text{fresh}}$. However, this difference is not noticeable by the simulated $\mathcal{A}$ by the CPA security of the AEAD scheme. Then, the real world and simulated real world behave the same, and parties in the real world and ideal world output the same things, which also holds for $\mathcal{A}$ and the forwarded simulated $\mathcal{A}$ in the ideal world. □

We can then use the UC Composition Theorem 2.8 to show the claim from the proof overview of Theorem 5.1 that there exists a protocol realizing $\mathcal{F}_{\text{fresh}}$ in the $\mathcal{F}_{\text{gsetup}}$-hybrid model.

Corollary A.3. *There exists a protocol $\hat{\pi}_{\text{fresh}}$ which realizes $\mathcal{F}_{\text{fresh}}$ in the $\mathcal{F}_{\text{gsetup}}$-hybrid model.*

Appendix B. Security Proof of Main Theorem

Having defined and realized $\mathcal{F}_{\text{fresh}}$, we are set up to prove our main result Theorem 5.1. For this, we first construct protocol $\pi_{\text{f-dsm}}$ which realizes $\mathcal{F}_{\text{dsm}}$ in the $\mathcal{F}_{\text{fresh}}$-hybrid model. The protocol $\pi_{\text{f-dsm}}$ is easier to describe than π_{dsm}, because functionality $\mathcal{F}_{\text{fresh}}$ abstracts away useful requirements on the communication channel. In particular $\mathcal{F}_{\text{fresh}}$ prevents the adversary from altering messages containing subkey permissions, reading confidential parts containing subkeys, or replaying messages. Since the following protocol $\pi_{\text{f-dsm}}$ is identical to π_{dsm} except for the encapsulation of communication into $\mathcal{F}_{\text{fresh}}$, we refer to Section 5 for an extensive description of $\pi_{\text{f-dsm}}$.

Protocol $\pi_{\text{f-dsm}}$

- Each party ignores any message from any party that contains some session ID sid until after they have received $(\texttt{init}, \mathsf{sid})$.
- If a set is accessed although it has not been initialized, it is first initialized as $\emptyset$.
- Memory is per party and sid.

Initialization $\mathcal{P}_j$ on input $(\texttt{init}, \mathsf{sid})$: Do nothing

Key Generation $\mathcal{P}_j$ on input $(\texttt{keygen}, \mathsf{sid}, \mathcal{H})$

1: If $\mathcal{P}_j \notin \mathcal{H}$, output $\perp$.
2: Compute $((\mathsf{sk}_i)_{i\in[\ell_{\text{sub}}]}, \mathsf{pk}) \xleftarrow{\$} \mathsf{KeyGen}(1^\lambda)$.
3: Set $\mathcal{B}^{\mathsf{pk}}_j \leftarrow [\ell_{\text{sub}}]$.
4: Store $(\texttt{key}, \mathsf{sid}, \mathcal{H}, (\mathsf{sk}_i)_{i\in[\ell_{\text{sub}}]}, \mathsf{pk})$.
5: Output $(\texttt{pk}, \mathsf{sid}, \mathsf{pk}, \ell_{\text{sub}})$.

Notify $\mathcal{P}_j$ on input $(\texttt{notify}, \mathsf{sid}, \mathsf{pk}, \mathcal{P}_t)$

6: If $(\texttt{key}, \mathsf{sid}, \mathcal{H}, (\mathsf{sk}_i)_{i\in[\ell^{\mathsf{pk}}_{\text{sub}}]}, \mathsf{pk})$ for some $\mathcal{H}, (\mathsf{sk}_i)_{i\in[\ell^{\mathsf{pk}}_{\text{sub}}]}$, continue, else output $\perp$.
7: If $\mathcal{P}_j \notin \mathcal{H}$ or $\mathcal{P}_t \notin \mathcal{H}$, output $\perp$.
8: Set $h \leftarrow (\texttt{key}, \mathsf{pk})$, $\mu = (\mathsf{sk}_i)_{i\in[\ell_{\text{sub}}]}$ and send $(\texttt{send}, \mathsf{sid}, \mathcal{H}, \mathcal{P}_t, h, \mu)$ to $\mathcal{F}_{\text{fresh}}$.

Notified $\mathcal{P}_t$ on input $(\texttt{dlvr}, \mathsf{sid}, \mathcal{H}, \mathcal{P}_j, h, \mu)$ with $h = (\texttt{key}, \mathsf{pk})$ from $\mathcal{F}_{\text{fresh}}$

9: Parse $(\mathsf{sk}_i)_{i\in[\ell_{\text{sub}}]} \leftarrow \mu$.
10: Store $(\texttt{key}, \mathsf{sid}, \mathcal{H}, (\mathsf{sk}_i)_{i\in[\ell_{\text{sub}}]}, \mathsf{pk})$.

Assign $\mathcal{P}_j$ on input $(\texttt{assign}, \mathsf{sid}, \mathsf{pk}, \mathcal{P}_t, \mathsf{req})$

11: If $(\texttt{key}, \mathsf{sid}, \mathcal{H}, \cdot, \mathsf{pk})$ exists in memory for some $\mathcal{H}$, continue, else output $\perp$.
12: If $\mathcal{P}_t \notin \mathcal{H}$, output $\perp$.
13: If $\mathsf{req} \notin \mathcal{B}_j^{\mathsf{pk}}$, output $\perp$.
14: Set $\mathcal{B}_j^{\mathsf{pk}} \xleftarrow{\setminus} \mathsf{req}$.
15: Set $h \leftarrow (\texttt{assgnd}, \mathsf{pk}, \mathsf{req})$, $\mu \leftarrow \top$ and send $(\texttt{send}, \mathsf{sid}, \mathcal{H}, \mathcal{P}_t, h, \mu)$ to $\mathcal{F}_{\text{fresh}}$.

Assigned $\mathcal{P}_t$ on input $(\texttt{dlvr}, \mathsf{sid}, \mathcal{H}, \mathcal{P}_j, h, \mu)$ with $h = (\texttt{assgnd}, \mathsf{pk}, \mathsf{req})$ from $\mathcal{F}_{\text{fresh}}$

16: If $(\texttt{key}, \mathsf{sid}, \cdot, \cdot, \mathsf{pk})$ exists in memory for some $\mathcal{H}$, continue, else output $\perp$.
17: Set $\mathcal{B}_t^{\mathsf{pk}} \xleftarrow{\cup} \mathsf{req}$.
18: Output $(\texttt{keyset}, \mathsf{sid}, \mathsf{pk}, \mathcal{B}_t^{\mathsf{pk}})$.

Sign $\mathcal{P}_j$ on input $(\texttt{sign}, \mathsf{sid}, \mathsf{pk}, \mu, l)$

19: If $(\texttt{key}, \mathsf{sid}, \cdot, (\mathsf{sk}_i)_{i \in [\ell_{\text{sub}}]}, \mathsf{pk})$ is not in memory, output $\perp$.
20: If $l \notin \mathcal{B}_j^{\mathsf{pk}}$, output $\perp$.
21: Set $\mathcal{B}_j^{\mathsf{pk}} \xleftarrow{\setminus} \{l\}$.
22: Compute $\sigma \xleftarrow{\$} \mathsf{Sign}(1^\lambda, \mathsf{sk}_l, \mu)$ and output σ.

Verify $\mathcal{P}$ on input $(\texttt{vrfy}, \mathsf{sid}, \mathsf{pk}, \mu, \sigma)$

23: Compute $b \xleftarrow{\$} \mathsf{Vrfy}(1^\lambda, \mathsf{pk}, \mu, \sigma)$.
24: Output $(\texttt{verified}, \mathsf{sid}, \mathsf{pk}, \mu, \sigma, b)$.

Theorem B.1. *Protocol $\pi_{\text{f-dsm}}$ realizes functionality $\mathcal{F}_{\text{dsm}}$ in the $\mathcal{F}_{\text{fresh}}$-hybrid model if the signature scheme with subkeys $\Sigma = (\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Vrfy})$ is strong EUF-CMA one-time secure (Definition 2.2) and linear-time (Definition 2.3).*

Before proving Theorem B.1, we restate our main result, Theorem 5.1, in the following corollary. Recall that Theorem 5.1 informally claimed that π_{dsm} realizes $\mathcal{F}_{\text{dsm}}$, without explicitly referring to the $\mathcal{F}_{\text{gsetup}}$-hybrid model. The corollary below makes this precise. The result follows by applying the UC composition theorem 2.8 on the protocols from Theorem B.1 and Corollary A.3.

Corollary B.2. *Protocol π_{dsm} realizes functionality $\mathcal{F}_{\text{dsm}}$ in the $\mathcal{F}_{\text{gsetup}}$-hybrid model, if the signature scheme with subkeys Σ is strong EUF-CMA one-time secure (Definition 2.2) and linear-time (Definition 2.3) and if the AEAD scheme Π is CPA secure (Definition 2.5) and unforgeable (Definition 2.6).*

Proof. By Corollary A.3, there exists protocol $\hat{\pi}_{\text{fresh}}$ realizing $\mathcal{F}_{\text{fresh}}$ in the $\mathcal{F}_{\text{gsetup}}$-hybrid model. By Theorem B.1, $protdsmfresh$ realizes functionality $\mathcal{F}_{\text{dsm}}$ in the $\mathcal{F}_{\text{fresh}}$-hybrid model. Then, by the UC composition theorem 2.8 $\pi_{\text{f-dsm}}^{\hat{\pi}_{\text{fresh}}}$ realizes $\mathcal{F}_{\text{dsm}}$ in the $\mathcal{F}_{\text{gsetup}}$-hybrid model. Since $\pi_{\text{f-dsm}}^{\pi_{\text{fresh}}}$ and π_{dsm} are identical, the result follows. □

We continue to prove Theorem B.1. For a higher-level overview on the proof structure, we refer to the outline after Theorem 5.1.

The proof is very similar to the proof of Lemma 3.5 in [5]. On the one hand, however, we have three simplifying properties. First, because Vrfy is deterministic we do not need to assume computational consistency of verifications. Second, because we assume perfect correctness, some of the hybrids are perfectly indistinguishable instead of computationally indistinguishable as in the proof of [5]. Third, in our setting parties are incorruptible. Hence, the simulator does not need to come up with persuasive views of corrupted parties, containing, for example, the randomness used to sample signatures. On the other hand, the use of subkeys for signing as well as the communication for transferring knowledge and permissions of subkeys add an additional layer of complexity.

Proof of Theorem B.1. To prove that $\pi_{\text{f-dsm}}$ realizes $\mathcal{F}_{\text{dsm}}$ in the $\mathcal{F}_{\text{fresh}}$-hybrid model, we need to show that for any ppt adversary $\mathcal{A}$ there exists a simulator $\mathcal{S}_{\text{dsm}}$ such that for all ppt environments $\mathcal{E}$

$$\{\text{REAL}_{\pi_{\text{f-dsm}}, \mathcal{A}, \mathcal{E}}(\kappa, z)\}_{\kappa \in \mathbb{N}, z \in \{0,1\}^*} \approx_{\text{c}} \{\text{IDEAL}_{\mathcal{F}_{\text{dsm}}, \mathcal{S}_{\text{dsm}}, \mathcal{E}}(\kappa, z)\}_{\kappa \in \mathbb{N}, z \in \{0,1\}^*}$$

Before constructing simulator $\mathcal{S}_{\text{dsm}}$, we transform the real world experiment into the ideal world experiment step by step. Concretely, we create hybrid experiments by adding code from $\mathcal{F}_{\text{dsm}}$ to the real world experiment and providing the protocol participants access to global memory. In each step, we first describe the additions or changes to the previous hybrid $\mathcal{H}_{\text{i}}$. Then, we prove that for any ppt environment, the resulting $\mathcal{H}_{\text{i+1}}$ is computationally or perfectly indistinguishable from $\mathcal{H}_{\text{i}}$. For illustration, we describe the final protocol associated to hybrid $\mathcal{H}_{13}$ in Figure B.

Hybrid $\mathcal{H}_0$: This hybrid equals the real world experiment with protocol $\pi_{\text{f-dsm}}$, environment $\mathcal{E}$, and adversary $\mathcal{A}$.

Hybrid $\mathcal{H}_1$: Add the variables $\ell_{\mathsf{pk}}, \ell_{\mathsf{sign}} \leftarrow 1$ and $\mathcal{K}_{\mathsf{pk}}, \mathcal{K}_{\mathsf{sign}}, \mathcal{K}_{\mathsf{send}} \leftarrow \emptyset$ from **Initialization** in Step 1 to global memory and update them as in $\mathcal{F}_{\text{dsm}}$. Add variable rmode, whose value remains fixed to 0. Also add all code from $\mathcal{F}_{\text{dsm}}$ which is run in case of $\mathsf{rmode} = 1$.
Because setting the variables and fixing rmode to 0 is internal and not observable by the environment $\mathcal{E}$, $\mathcal{H}_0 \equiv \mathcal{H}_1$ follows.

Hybrid $\mathcal{H}_2$: When $\mathcal{E}$ inputs $(\texttt{sign}, \mathsf{sid}, \mathsf{pk}, \mu, l)$ to $\mathcal{P}$ *and* $\mathcal{P}$ outputs signature σ, then store $(\texttt{sig}, \mathsf{sid}, \mathsf{pk}, \mu, \sigma)$ in global memory.
Again, this change is purely *internal* and not noticeable by $\mathcal{E}$, hence $\mathcal{H}_1 \equiv \mathcal{H}_2$.

We require following uniqueness Lemma, analogous to Lemma 3.3 from [5] except for our syntax with subkeys.

Lemma B.3 (Unique public keys)**.** *If $\Sigma = (\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Vrfy})$ is a strong EUF-CMA signature scheme with subkeys (Definition 2.2), then for any ppt adversary $\mathcal{A}$ there exists a negligible function* negl *such that for all security parameters λ*

$$\Pr[\mathsf{pk} \in \mathcal{K}_{\mathsf{pk}} : \mathcal{K}_{\mathsf{pk}} \xleftarrow{\$} \mathcal{A}(1^\lambda), (\cdot, \mathsf{pk}) \xleftarrow{\$} \mathsf{KeyGen}(1^\lambda)] \leq \mathsf{negl}(\lambda).$$

Intuitively, Lemma B.3 states that given a polynomially-sized set of adversarially chosen public keys, KeyGen outputs a key from this set with negligible probability. This facilitates proving indistinguishability of following hybrid from $\mathcal{H}_2$. For the proof of Lemma B.3 we refer to the detailed proof of Lemma 3.3 in [5] which is analogous.

Hybrid $\mathcal{H}_3$: Add the public key uniqueness check from **Key Generation** in Step 6, i.e. if the sampled $\mathsf{pk} \in \mathcal{K}_{\mathrm{pk}}$, set $\mathsf{rmode} \leftarrow 1$.
The only difference to $\mathcal{H}_2$ occurs, if $\mathsf{rmode} \leftarrow 1$ is set. By a straightforward reduction to Lemma B.3 this only happens with negligible probability. In particular, the reduction simulates $\mathcal{H}_3$ internally and outputs $\mathcal{K}_{\mathrm{pk}}$ if $\mathsf{rmode} \leftarrow 1$ is set. Hence, we have $\mathcal{H}_2 \approx_{\mathrm{c}} \mathcal{H}_3$.

Hybrid $\mathcal{H}_4$: Add the behavior of $\mathcal{F}_{\text{dsm}}$ for key notifications using the indicator variables $\mathcal{N}_j^{\mathsf{pk}}$ and global memory. Also, instead of having the parties send concrete subkeys via $\mathcal{F}_{\text{fresh}}$, they only send dummy bitstrings of identical length.
In particular, after $\mathcal{P}_j$ generates keys $((\mathsf{sk}_i)_{i\in[\ell_{\text{sub}}]}, \mathsf{pk})$ in **Key Generation**, set $\mathcal{N}_j^{\mathsf{pk}} \leftarrow 1$ and store $(\mathtt{key}, \mathsf{sid}, \mathcal{H}, (\mathsf{sk}_i)_{i\in[\ell_{\text{sub}}^{\mathsf{pk}}]}, \mathsf{pk})$ in global memory. Next, add Step 13 and Step 14 from **Notify**, i.e. when run on input $(\mathtt{notify}, \mathsf{sid}, \mathsf{pk}, \mathcal{P}_t)$, if $\mathcal{N}_j^{\mathsf{pk}} = 0$, then output $\perp$. This is additional to the conditions $\mathcal{P}_j \notin \mathcal{H}$ or $\mathcal{P}_t \notin \mathcal{H}$, which are already part of $\pi_{\text{f-dsm}}$. Else, if $\mathcal{N}_j^{\mathsf{pk}} = 1$, store $(\mathtt{notified}, \mathsf{sid}, \mathsf{pk}, \mathcal{P}_j, \mathcal{P}_t)$ in global memory and fix the header to be sent as $\mu \leftarrow (0^{\mathcal{L}(i,\mathsf{pk})})_{i\in[\ell_{\text{sub}}]}$. This replaces the subkeys by bitstrings of 0s of identical length, since $\mathcal{L}(i, \mathsf{pk})$ denotes the length of the i-th subkey sk_i of pk (cf. Definition 2.1). Also, add Step 18 from **Notified**. Thus, when some party $\mathcal{P}_t$ is run on $(\mathtt{dlvr}, \mathsf{sid}, \mathcal{H}, \mathcal{P}_j, h, \mu)$ with $h = (\mathtt{key}, \mathsf{pk})$ from $\mathcal{F}_{\text{fresh}}$ and was already initialized, if $(\mathtt{notified}, \mathsf{sid}, \mathsf{pk}, \mathcal{P}_j, \mathcal{P}_t)$ is in global memory, then set $\mathcal{N}_t^{\mathsf{pk}} = 0$ Finally, when the **Assigned** or **Sign** interfaces are run on some pk, if $\mathcal{N}_j^{\mathsf{pk}} = 0$, then output $\perp$.
The view of any adversary $\mathcal{A}$ in $\mathcal{H}_4$ is identical to $\mathcal{H}_3$ due to two properties. First, because $\mathcal{F}_{\text{fresh}}$ only reveals the *length* of a sent message μ to $\mathcal{A}$ and $\sum_{i\in[\ell_{\text{sub}}]}|0^{\mathcal{L}(i,\mathsf{pk})}| = \sum_{i\in[\ell_{\text{sub}}]}|\mathsf{sk}_i|$ holds. Second, because $\mathcal{N}_j^{\mathsf{pk}} \leftarrow 1$ is set in $\mathcal{H}_4$ if and only if the subkeys become known to party $\mathcal{P}_j$ in **Key Generation** or **Notified** as in $\mathcal{H}_3$. Thus, $\mathcal{H}_3 \equiv \mathcal{H}_4$.

By the changes of $\mathcal{H}_4$, the interfaces for **Notify** and **Notified** become independent of the concrete secret subkeys. Consequently, in all subsequent reductions, simulating these interfaces is straightforward, and we omit explicitly mentioning this.

Before continuing with $\mathcal{H}_5$, we introduce another helpful Lemma. This Lemma is strongly connected to the *subkey partition* property and guarantees that each subkey is used at most once for signing in $\mathcal{H}_4$ and subsequent real world hybrids. Conditioned on the event that all honestly generated keys are unique (Lemma B.3), which holds with overwhelming probability, we prove Lemma B.4 against any, even inefficient, environment and adversary.

Lemma B.4 (One-time subkey usage)**.** *After an execution of $\mathcal{H}_4$, denote by Q^{pk} the set $\{(c, \ell) | c$-th signature for pk sampled with index $\ell\}$ of indices of used subkeys. Then for all environments $\mathcal{E}$, all adversaries $\mathcal{A}$, and after all executions of $\mathcal{H}_4$ where KeyGen does not generate $\mathsf{pk} \in \mathcal{K}_{\mathrm{pk}}$, there is no $(c, \ell), (c', \ell') \in Q^{\mathsf{pk}}$ such that $c \neq c'$ and $\ell = \ell'$.*

We postpone the invariant-based proof of Lemma B.4 to the end of this section. Lemma B.4 is crucial in theory and practice, since in case of multi-time usage of one-time subkeys, unforgeability cannot be guaranteed anymore. Though we only prove Lemma B.4 explicitly for $\mathcal{H}_4$, it applies to each subsequent hybrid because they handle storing, transferring, and using subkey permissions identically. Intuitively, it holds because i) signing permissions are only re-assigned if they were removed by the previous owner and ii) functionality $\mathcal{F}_{\text{fresh}}$ ensures that assignment messages are delivered *at most* once.

Hybrid $\mathcal{H}_5$: Add the first constraint of Step 38 from **Sign** ensuring that two messages cannot have the same signature. Concretely, when $(\mathtt{sign}, \mathsf{sid}, \mathsf{pk}, \mu, l)$ is queried, if $\mathsf{rmode} = 0$ and there is an entry $(\mathtt{sig}, \mathsf{sid}, \mathsf{pk}, \mu', \sigma')$ in memory such that $\mu' \neq \mu$ and $\mathsf{Vrfy}(1^\lambda, \mathsf{pk}, \mu, \sigma') = 1$, then set $\mathsf{rmode} \leftarrow 1$.
An environment may only distinguish if the new checks pass and $\mathsf{rmode} \leftarrow 1$ is set. This happens with negligible probability, as we prove by reduction to *strong* EUF-CMA one-time security (Definition 2.2) of SwS Σ. Specifically, fix a ppt adversary $\mathcal{A}$ and a ppt environment $\mathcal{E}$ that generates up to q_{pk} public keys using interface **Key Generation**, which distinguishes $\mathcal{H}_4$ and $\mathcal{H}_5$ with advantage ϵ. We construct a forger $\mathcal{F}$ against Σ which emulates $\mathcal{H}_5$ towards $\mathcal{E}$ using its signature oracle SigO. Initially, $\mathcal{F}$ samples index $i \xleftarrow{\$} [q_{\mathsf{pk}}]$. On the i-th **Key Generation** request by $\mathcal{E}$ within the emulation, forger $\mathcal{F}$ provides its challenge public key pk instead of generating keys honestly. Then, whenever some party $\mathcal{P}$ would generate a signature on μ for pk after $\mathcal{E}$ requested $(\mathtt{sign}, \mathsf{sid}, \mathsf{pk}, \mu, l)$, $\mathcal{F}$ checks if for any $\mu' \neq \mu$ an entry $(\mathtt{sig}, \mathsf{sid}, \mathsf{pk}, \mu', \sigma')$ is in memory such that $\mathsf{Vrfy}(1^\lambda, \mathsf{pk}, \mu, \sigma') = 1$. If yes, $\mathcal{F}$ outputs (μ, σ') as a forgery attempt. Otherwise it queries $\sigma \xleftarrow{\$} \mathsf{SigO}(l, \mu)$ and returns σ to $\mathcal{P}$. Note, SigO always answers with a signature σ, because each index l is used at most once for signing by Lemma B.4. Therefore, forger $\mathcal{F}$ is well-defined.
We relate the success probability ϵ' of $\mathcal{F}$ and the distinguishing advantage ϵ of $\mathcal{E}$ in two steps. First, we show that $\mathcal{F}$ only outputs *valid* forgeries. Second, we argue that $\mathcal{F}$ *always* outputs a forgery if $\mathsf{rmode} \leftarrow 1$ is set for the sid associated to the i-th public key, i.e. the challenge

public key pk. These two steps and the random choice of index i from $[q_{\mathsf{pk}}]$ imply $\epsilon' > \epsilon/q_{\mathsf{pk}}$. Since ϵ' is negligible by strong EUF-CMA one-time security and q_{pk} is polynomial, $\mathcal{H}_4 \approx_c \mathcal{H}_5$ follows.

For step one, note that if $\mathcal{F}$ outputs (μ, σ'), then for some $\mu' \neq \mu$ an entry $(\texttt{sig}, \mathsf{sid}, \mathsf{pk}, \mu', \sigma')$ is in memory such that $\mathsf{Vrfy}(\mathsf{pk}, \mu, \sigma') = 1$. This holds by construction of $\mathcal{F}$ and determinism of Vrfy. Although the signature σ' is valid for μ, it is still necessary to show that σ' was not generated previously on μ by SigO, as required by the *strong* EUF-CMA game. For the sake of contradiction assume it was. We make a case distinction on the memory before SigO returns σ' on μ for the first time. As first case, assume entry $(\texttt{sig}, \mathsf{sid}, \mathsf{pk}, \mu', \sigma')$ was in memory. Then $\mathcal{F}$ would have found σ' and output the valid forgery (μ, σ') instead of querying SigO on μ. As second case, assume entry $(\texttt{sig}, \mathsf{sid}, \mathsf{pk}, \mu', \sigma')$ was not in memory. The entry, however, has to be stored before $\mathcal{F}$ outputs forgery (μ, σ'), because otherwise $\mathcal{F}$ would not output it. Also, the entry is stored after $(\texttt{sig}, \mathsf{sid}, \mathsf{pk}, \mu, \sigma')$, since this is stored immediately after the current query on μ. Lastly, the entry is only stored when a signature on message μ' is queried. But when μ' is queried and $(\texttt{sig}, \mathsf{sid}, \mathsf{pk}, \mu, \sigma')$ is already stored, $\mathcal{F}$ would find and output the valid forgery (μ', σ') without querying SigO and stop. Both cases lead to a contradiction, because $\mathcal{F}$ would have stopped earlier with its forgery attempt. Hence, σ' was never generated on μ by SigO and the forgery (μ, σ') makes $\mathcal{F}$ win the strong EUF-CMA game.

For step two, note that $\mathcal{F}$ simulates $\mathcal{H}_5$ towards $\mathcal{E}$ perfectly and independently of index i until it stops. Additionally, as stated above, $\mathcal{E}$ may only gain its advantage ϵ of distinguishing $\mathcal{H}_4$ and $\mathcal{H}_5$ if the added checks pass. If these checks pass *for the* sid *associated to the i-th public key*, then $\mathcal{F}$ outputs a forgery, which is valid as argued in step one. Because i is chosen uniformly at random and simulation is perfect, this gives a lower bound on the forging probability ϵ' of at least ϵ/q_{pk}. Because ϵ' is negligible by assumption on Σ and q_{pk} is polynomial, the distinguishing advantage ϵ is negligible as well. This implies $\mathcal{H}_4 \approx_c \mathcal{H}_5$.

Hybrid $\mathcal{H}_6$: Continue implementing Step 38 from **Sign**. Instead of immediately setting $\mathsf{rmode} \leftarrow 1$ when there is $(\texttt{sig}, \mathsf{sid}, \mathsf{pk}, \mu', \sigma')$ with $\mu \neq \mu'$ and $\mathsf{Vrfy}(1^\lambda, \mathsf{pk}, \mu, \sigma') = 1$, wait for $\mathcal{P}$ to output a signature σ. Only if $\sigma = \sigma'$ set $\mathsf{rmode} \leftarrow 1$ and otherwise do nothing.

The conditions of $\mathcal{H}_6$ which trigger $\mathsf{rmode} \leftarrow 1$ are a subset of $\mathcal{H}_5$, therefore $\mathcal{H}_4 \approx_c \mathcal{H}_5$ implies $\mathcal{H}_4 \approx_c \mathcal{H}_6$.

Hybrid $\mathcal{H}_7$: Continue implementing Step 38 from **Sign**, by preparing detection of bad signatures. Particularly, on $(\texttt{sign}, \mathsf{sid}, \mathsf{pk}, \mu, l)$ from $\mathcal{E}$ first wait for $\mathcal{P}$ to output signature σ, then run the checks from $\mathcal{H}_6$. *Additionally*, if $\mathsf{Vrfy}(1^\lambda, \mathsf{pk}, \mu, \sigma) = 0$, set $\mathsf{rmode} \leftarrow 1$.

Since this re-ordering is not observable and since by perfect correctness of Σ verification never outputs 0 on an honestly generated signature, we have $\mathcal{H}_6 \equiv \mathcal{H}_7$.

Hybrid $\mathcal{H}_8$: Implement Step 44 in **Verify**, i.e. when $\mathcal{E}$ sends $(\texttt{vrfy}, \mathsf{sid}, \mathsf{pk}, \mu, \sigma)$ to some party $\mathcal{P}$ and $(\texttt{sig}, \mathsf{sid}, \mathsf{pk}, \mu, \sigma)$ already exists in memory, then $\mathcal{P}$ fixes $b \leftarrow 1$ instead of using the output of Vrfy.

This only differs from $\mathcal{H}_8$, if Vrfy would output 0 on an honestly generated signature, because up to now a signature σ is only stored if it was output by Sign. Due to perfect correctness this never happens and $\mathcal{H}_7 \equiv \mathcal{H}_8$ follows.

Hybrid $\mathcal{H}_9$: Implement Step 44, Step 48, and Step 49 from **Verify** ensuring consistency of verification results. Concretely, when $\mathcal{E}$ sends $(\texttt{vrfy}, \mathsf{sid}, \mathsf{pk}, \mu, \sigma)$ to some party $\mathcal{P}$, if entry $(\texttt{badsig}, \mathsf{sid}, \mathsf{pk}, \mu, \sigma)$ is in memory then set $b \leftarrow 0$. Else, run $b \leftarrow \mathsf{Vrfy}(1^\lambda, \mathsf{pk}, \mu, \sigma)$ and in case $b = 0$ store $(\texttt{badsig}, \mathsf{sid}, \mathsf{pk}, \mu, \sigma)$ while in case $b = 1$ store $(\texttt{sig}, \mathsf{sid}, \mathsf{pk}, \mu, \sigma)$.

Recall that when $\mathcal{E}$ queries verification of signature σ on message μ *after* obtaining them as a **Sign** result, then $\mathcal{H}_8$ fixes the outcome to 1 and the new checks are not run. Hence, $\mathcal{H}_9$ may only differ from $\mathcal{H}_8$ if verification for signature σ and message μ was queried twice and σ was not generated by running **Sign** on μ. While in $\mathcal{H}_9$ both results are consistent by definition, in $\mathcal{H}_8$ they depend on the outcomes of Vrfy. However, since Vrfy is deterministic, both outcomes are the same, hence $\mathcal{H}_8 \equiv \mathcal{H}_9$.

Hybrid $\mathcal{H}_{10}$: Implement the second condition from Step 38 in **Sign**: When party $\mathcal{P}$ computes $\sigma \xleftarrow{\$} \mathsf{Sign}(1^\lambda, \mathsf{sk}_l, \mu)$ and if $(\texttt{badsig}, \mathsf{sid}, \mathsf{pk}, \sigma, \mu)$ is in memory, then instead of checking $\mathsf{Vrfy}(1^\lambda, \mathsf{pk}, \mu, \sigma) = 0$ as introduced in $\mathcal{H}_7$, immediately set $\mathsf{rmode} \leftarrow 1$.

Previously, we stored $(\texttt{badsig}, \mathsf{sid}, \mathsf{pk}, \sigma, \mu)$ only if a verification query on μ and σ failed *and* no previous **Sign** query on μ returned σ. The latter held because since $\mathcal{H}_8$ running **Verify** on signatures generated in **Sign** always returned 1. Hence, this hybrid only differs from $\mathcal{H}_9$ if a first verification on μ and σ fails, thus adding a `badsig` entry, then Sign on μ outputs σ, and running Vrfy afterwards succeeds. By deterministic verification this never happens, therefore $\mathcal{H}_9 \equiv \mathcal{H}_{10}$.

Hybrid $\mathcal{H}_{11}$: Effectively we implement Step 46 from **Verify**. When environment $\mathcal{E}$ sends $(\texttt{vrfy}, \mathsf{sid}, \mathsf{pk}, \mu, \sigma)$ to some party $\mathcal{P}$, if there is an entry $(\texttt{key}, \mathsf{sid}, \cdot, \cdot, \mathsf{pk})$ and no entry $(\texttt{sig}, \mathsf{sid}, \mathsf{pk}, \mu, \sigma)$, then behave as if Vrfy outputs 0 instead of actually running Vrfy. Especially, as defined in $\mathcal{H}_9$, a `badsig` entry is stored.

This hybrid differs only if for some honestly generated pk the environment sends a message μ

and signature σ under pk such that Vrfy outputs 1 before **Sign** outputs σ on μ. The reduction to (strong) EUF-CMA security of SwS (Definition 2.2) using an environment which makes q_{pk} **Key Generation** queries works as follows. Forger $\mathcal{F}$ samples index $i \xleftarrow{\$} [q_{\mathsf{pk}}]$ and then simulates $\mathcal{H}_{11}$ where the i-th public key is replaced by the challenge public key pk. The signature oracle SigO is used to answer **Sign** queries corresponding to pk. In particular, whenever any emulated party $\mathcal{P}$ would run $\mathsf{Sign}(1^\lambda, \mathsf{sk}_l, \mu)$, fix $\sigma \xleftarrow{\$} \mathsf{SigO}(\mu, l)$ as outcome instead. Whenever $\mathcal{E}$ requests verification under pk for a pair of message μ and signature σ which was not generated by **Sign** before, use Vrfy to check the pair's validity. If it is valid, output forgery (μ, σ).
For analysis, recall that Lemma B.4 guarantees that each subkey index is at most used once for signing. Hence, all queries from $\mathcal{F}$ to SigO are correctly answered. Because verification is deterministic, the forger's check outputs 1 if and only if Vrfy outputs 1 in $\mathcal{H}_{11}$. Hence, whenever the environment is able to distinguish, the forger outputs the valid forgery of message μ and signature σ. By strong EUF-CMA security this only happens with negligible probability, hence $\mathcal{H}_{10} \approx_c \mathcal{H}_{11}$.

Hybrid $\mathcal{H}_{12}$: When $\sigma \xleftarrow{\$} \mathsf{Sign}(1^\lambda, \mathsf{sk}_l, \mu)$ is run in **Sign** and no entry $(\texttt{badsig}, \mathsf{sid}, \mathsf{pk}, \mu, \sigma)$ is in memory, then instead of running Vrfy behave as if it outputted 1. Hence Vrfy, introduced in hybrid $\mathcal{H}_7$, is not run anymore after signing.
This hybrid only differs if Vrfy would output 0 on an honestly generated signature. Due to perfect correctness this never happens, thus $\mathcal{H}_{11} \equiv \mathcal{H}_{12}$.

Hybrid $\mathcal{H}_{13}$: In this hybrid the algorithms of $(\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Vrfy})$ of Σ are replaced by algorithms $(\mathsf{KeyGen}', \mathsf{Sign}', \mathsf{Vrfy}')$, which have the security parameter λ hard-coded. Additionally, by adding no-op operations, they are padded to a suitable description length T. This length T is chosen such that both, it is bigger than the number of computational steps $\mathsf{KeyGen}(1^\lambda)$ requires and $T \cdot (|\mu| + 1)$ is bigger than the number of computational steps to run either Sign or Vrfy on message μ.
Because the scheme is linear-time (Definition 2.3), this padding is possible. As these changes are internal and not observable by any environment $\mathcal{E}$, we have $\mathcal{H}_{12} \equiv \mathcal{H}_{13}$.

Finally, we bridge hybrid experiment $\mathcal{H}_{13}$, which by transitivity is computationally indistinguishable from the real world experiment $\mathcal{H}_0$, to the ideal world experiment. Recall, for any fixed real world adversary $\mathcal{A}$, which is part of $\mathcal{H}_{13}$, we need to construct a simulator $\mathcal{S}_{\text{dsm}}$ such that for any ppt environment $\mathcal{E}$ the ideal world experiment with $\mathcal{S}_{\text{dsm}}$ and $\mathcal{F}_{\text{fresh}}$ is not distinguishable from $\mathcal{H}_{13}$. In particular, simulator $\mathcal{S}_{\text{dsm}}$ on 1^λ works as follows.

1) Internally set up a copy of adversary $\mathcal{A}$, named $\mathcal{A}^{\mathsf{sim}}$, and of $\mathcal{F}_{\text{fresh}}$, named $\mathcal{F}_{\text{fresh}}^{\mathsf{sim}}$, and connect them with each other as $\mathcal{A}$ and $\mathcal{F}_{\text{fresh}}$ in $\mathcal{H}_{13}$. Additionally connect $\mathcal{A}^{\mathsf{sim}}$ to $\mathcal{E}$ as in $\mathcal{H}_{13}$.
2) On input $(\texttt{init}, \mathsf{sid})$ from $\mathcal{F}_{\text{dsm}}$: If sid was sent the first time, send $(\texttt{algs}, \mathsf{sid}, \Sigma')$ to $\mathcal{F}_{\text{dsm}}$, where $\Sigma' = (\mathsf{KeyGen}', \mathsf{Sign}', \mathsf{Vrfy}')$ such that security parameter 1^λ is hard-coded into each original algorithm and their description length is padded by no-op operations as in $\mathcal{H}_{13}$.
3) On input $(\texttt{notified}, \mathsf{sid}, \mathcal{H}, \ell_{\text{sub}}, \mathsf{pk}, \mathcal{P}_j, \mathcal{P}_t)$ from $\mathcal{F}_{\text{dsm}}$: Set header $h \leftarrow (\texttt{key}, \mathsf{pk})$, message $\mu \leftarrow (0^{\mathcal{L}(i,\mathsf{pk})})_{i \in [\ell_{\text{sub}}]}$ and send $(\texttt{send}, \mathsf{sid}, \mathcal{H}, \mathcal{P}_t, h, \mu)$ to $\mathcal{F}_{\text{fresh}}^{\mathsf{sim}}$ on behalf of $\mathcal{P}_j$.
4) On input $(\texttt{assgnd}, \mathsf{sid}, \mathsf{pk}, \mathcal{P}_j, \mathcal{P}_t, \mathsf{req})$ from $\mathcal{F}_{\text{dsm}}$: Set header $h \leftarrow (\texttt{assgnd}, \mathsf{pk}, \mathsf{req})$, message $\mu \leftarrow \top$ and send $(\texttt{send}, \mathsf{sid}, \mathcal{H}, \mathcal{P}_t, h, \mu)$ to $\mathcal{F}_{\text{fresh}}^{\mathsf{sim}}$ on behalf of $\mathcal{P}_j$.
5) When $\mathcal{F}_{\text{fresh}}^{\mathsf{sim}}$ would send $(\texttt{dlvr}, \mathsf{sid}, \mathcal{H}, \mathcal{P}_j, h, \mu)$ to $\mathcal{P}_t$:
   a) If header $h = (\texttt{key}, \mathsf{pk})$, then send $(\texttt{notified}, \mathsf{sid}, \mathsf{pk}, \mathcal{P}_j, \mathcal{P}_t)$ to $\mathcal{F}_{\text{dsm}}$.
   b) If header $h = (\texttt{assgnd}, \mathsf{pk}, \mathsf{req})$, then send $(\texttt{assgnd}, \mathsf{sid}, \mathsf{pk}, \mathcal{P}_j, \mathcal{P}_t, \mathsf{req})$ to $\mathcal{F}_{\text{dsm}}$.

We now show that $\mathcal{H}_{13}$ is perfectly indistinguishable from the ideal world experiment run by $\mathcal{E}$, $\mathcal{S}_{\text{dsm}}$ and $\mathcal{F}_{\text{dsm}}$. By our hybrid transformations, the code of the protocol of $\mathcal{H}_{13}$ (cf. Figure B) and the code of the protocol of dummy parties and $\mathcal{F}_{\text{dsm}}$ are identical except for i) the termination checks for the algorithms of Σ' in $\mathcal{F}_{\text{dsm}}$ and ii) the communication with $\mathcal{F}_{\text{fresh}}$ in **Notify/Notified** and **Assign/Assigned** in $\mathcal{H}_{13}$. We now argue why $\mathcal{S}_{\text{dsm}}$ solves both cases.

Consider i) first. Since the signature scheme with subkeys Σ is linear-time (Definition 2.3), the simulator $\mathcal{S}_{\text{dsm}}$ can pad Σ to a suitable length T such that the termination checks in the ideal world experiment never fail. Hence, identically to $\mathcal{H}_{13}$, when running the algorithms of Σ', rmode is never set to 1 in the ideal world experiment.

Consider ii) next. We argue that the states of $\mathcal{A}^{\mathsf{sim}}$ and $\mathcal{F}_{\text{fresh}}^{\mathsf{sim}}$ are identical to the states $\mathcal{A}$ and $\mathcal{F}_{\text{fresh}}$ would have in $\mathcal{H}_{13}$. Due to this, in 5) $\mathcal{S}_{\text{dsm}}$ runs the **Notified** and **Assigned** interfaces of $\mathcal{F}_{\text{dsm}}$ exactly when their counterparts in $\mathcal{H}_{13}$ would be invoked.

First, consider 3) from $\mathcal{S}_{\text{dsm}}$, i.e. where it is activated on $(\texttt{notified}, \ldots)$. By definition of $\mathcal{F}_{\text{dsm}}$, this happens if and only if environment $\mathcal{E}$ previously sent $(\texttt{notify}, \mathsf{sid}, \mathsf{pk}, \mathcal{P}_t)$ to $\mathcal{P}_j$ and all sanity checks in **Notify** passed. Because in $\mathcal{H}_{13}$ the sanity checks in **Notify** are identical, these checks pass as well when $\mathcal{E}$ sends $(\texttt{notify}, \mathsf{sid}, \mathsf{pk}, \mathcal{P}_t)$ to $\mathcal{P}_j$. Especially, party $\mathcal{P}_j$ would reach the point where it sends header and message, computed identically as by $\mathcal{S}_{\text{dsm}}$ in 3), to $\mathcal{F}_{\text{fresh}}$. Hence, the view of $\mathcal{A}^{\mathsf{sim}}$ remains identical to the view of $\mathcal{A}$ in $\mathcal{H}_{13}$.

For 4) from $\mathcal{S}_{\text{dsm}}$ the argument is analogous: The checks in both interfaces for **Assign** are the same, thus in 4) simulator $\mathcal{S}_{\text{dsm}}$ sends the same message to $\mathcal{F}_{\text{fresh}}$ as $\mathcal{P}_j$ in $\mathcal{H}_{13}$ would do. Again, the view of $\mathcal{A}^{\mathsf{sim}}$ remains identical to the view of $\mathcal{A}$ in $\mathcal{H}_{13}$.

It is left to show that if **Notified** or **Assigned** are activated in $\mathcal{H}_{13}$, then the simulator's calls to $\mathcal{F}_{\text{dsm}}$ to these interfaces have the same outcome. Recall, by

definition, $\mathcal{F}_{\text{fresh}}$ outputs messages i) unchanged and ii) at most once. Hence, in case of 5a) where $\mathcal{F}_{\text{fresh}}^{\text{sim}}$ sends $(\texttt{dlvr}, \mathsf{sid}, \mathcal{H}, \mathcal{P}_j, h, \mu)$ with $h = (\texttt{key}, \mathsf{pk})$ to $\mathcal{P}_t$, h was previously sent to $\mathcal{F}_{\text{fresh}}$ by $\mathcal{P}_j$ but has not been delivered yet. This means the ideal functionality $\mathcal{F}_{\text{dsm}}$ invoked simulator $\mathcal{S}_{\text{dsm}}$ which only happens after an entry $(\texttt{notified}, \mathsf{sid}, \mathsf{pk}, \mathcal{P}_j, \mathcal{P}_t)$ is stored in $\mathcal{F}_{\text{dsm}}$. Hence, running **Notified** by sending $(\texttt{notified}, \mathsf{sid}, \mathsf{pk}, \mathcal{P}_j, \mathcal{P}_t)$ to $\mathcal{F}_{\text{dsm}}$ leads to deleting this entry and results in $\mathcal{N}_t^{\mathsf{pk}} \leftarrow 1$. This is the identical outcome as in $\mathcal{H}_{13}$.

Case 5b) is analogous. Since we know that $h = (\texttt{assgnd}, \mathsf{pk}, \mathsf{req})$ must have been sent previously in 4) but was not yet delivered, the corresponding entry $(\texttt{assgnd}, \mathsf{sid}, \mathsf{pk}, \mathcal{P}_j, \mathcal{P}_t, \mathsf{req})$ is stored in $\mathcal{F}_{\text{dsm}}$'s memory. Thus, triggering **Assigned** by $(\texttt{assgnd}, \mathsf{sid}, \mathsf{pk}, \mathcal{P}_j, \mathcal{P}_t, \mathsf{req})$, deletes this entry and updates the set $\mathcal{B}_t^{\mathsf{pk}}$, i.e. has the identical outcome as in $\mathcal{H}_{13}$.

□

We conclude this section with the proof of Lemma B.4, which was used to prove Theorem B.1.

Proof of Lemma B.4. We prove Lemma B.4 by invariants which hold throughout the execution over the number i of activations of protocol participants. Note that during the real world experiment, all executions of functionality $\mathcal{F}_{\text{fresh}}$ take place immediately after some party $\mathcal{P}$ triggered the **Send** command or before some party $\mathcal{P}$ is activated due to the **Deliver** command. Therefore, in the following we consider these executions as part of the execution of $\mathcal{P}$.

We introduce some useful notation first. Denote by $F^{\mathsf{pk}}[i]$ the set of headers of form $(\texttt{assgnd}, \mathsf{pk}, \mathsf{req})$ stored in $\mathcal{F}_{\text{fresh}}$ before activation i of some protocol participant. By $Q^{\mathsf{pk}}[i]$ we denote the set of pairs (c, ℓ) of counter c and signature σ for public key pk before activation i. Lastly, by $\mathcal{B}_j^{\mathsf{pk}}[i]$ we denote the set $\mathcal{B}_j^{\mathsf{pk}}$ stored in $\mathcal{P}_j$ before activation i. We assume that non-initialized sets are always empty.

Now we formulate four invariants which hold before and after the i-th execution of any party $\mathcal{P}$ for all $\mathsf{pk} \in \mathcal{K}_{\mathsf{pk}}$. While the first invariant reflects the statement from Lemma B.4, the three latter, auxiliary ones imply the *subkey partition* property, i.e. that each subkey is stored in at most one location.

$$\forall (c, \ell), (c', \ell') \in Q^{\mathsf{pk}}[i] : c \neq c' \implies \ell \neq \ell' \quad (1)$$

$$(\cdot, \ell) \in Q^{\mathsf{pk}}[i] \implies \forall j : \ell \notin \mathcal{B}_j^{\mathsf{pk}}[i] \cup F^{\mathsf{pk}}[i] \quad (2)$$

$$\forall j : \mathcal{B}_j^{\mathsf{pk}}[i] \cap F^{\mathsf{pk}}[i] = \emptyset \quad (3)$$

$$\forall j \neq k : \mathcal{B}_j^{\mathsf{pk}}[i] \cap \mathcal{B}_k^{\mathsf{pk}}[i] = \emptyset \quad (4)$$

Before activation $i = 1$ all sets are empty and the invariants hold. Next, we show that assuming the invariants hold before activation i, then they hold after the execution, i.e. are satisfied before $i+1$. Therefore we do a complete case distinction on the interface on which the activation of protocol participant $\mathcal{P}_j$ took place, which finishes the proof.

Initialization, Notify, Notfied, Verification: No changes to the sets, i.e. all properties remain satisfied.

Key Generation: Only (4) is affected by $\mathcal{B}_j^{\mathsf{pk}} \leftarrow [\ell_{\text{sub}}]$. Since by assumption all generated public keys are unique, for all parties $j \neq k$ we have $\mathcal{B}_k^{\mathsf{pk}}[i] = \mathcal{B}_k^{\mathsf{pk}}[i+1] = \emptyset$, thus $\mathcal{B}_j^{\mathsf{pk}}[i+1] \cap \mathcal{B}_k^{\mathsf{pk}}[i+1] = \emptyset$ is trivially satisfied.

Assign: This activation has effects only if $\mathsf{req} \subseteq \mathcal{B}_j^{\mathsf{pk}}$, as then all indices from req are removed from $\mathcal{B}_j^{\mathsf{pk}}$ and added to $F^{\mathsf{pk}}[i]$. Also recall, that we assume the subsequent activation of $\mathcal{F}_{\text{fresh}}$ as part of the activation of $\mathcal{P}_j$.
Clearly, (1) is maintained, as it is only affected in **Sign**. Since we only remove req at $\mathcal{P}_j$ and add it to $\mathcal{F}_{\text{fresh}}$, $\mathcal{B}_j^{\mathsf{pk}}[i] \cup F^{\mathsf{pk}}[i]$ equals $\mathcal{B}_j^{\mathsf{pk}}[i+1] \cup F^{\mathsf{pk}}[i+1]$, thus (2) is maintained. By the same argument (3) is maintained for $\mathcal{P}_j$ as well. By (4), for any party $\mathcal{P}_j \neq \mathcal{P}_k$ before step i $\mathsf{req} \not\subseteq \mathcal{B}_j^{\mathsf{pk}}[i]$ holds. Hence, adding req to $F^{\mathsf{pk}}[i]$ does not affect (3). Finally, (4) is maintained as we only decrease set $\mathcal{B}_j^{\mathsf{pk}}[i]$.

Assigned: This case is completely analogous to **Assign** with reversed roles, i.e. with removing req from $F^{\mathsf{pk}}[i]$ and adding them to $\mathcal{B}_j^{\mathsf{pk}}[i]$.

Sign: Note, only if the requested subkey's index ℓ is in $\mathcal{B}_j^{\mathsf{pk}}[i]$, the c-th signature is computed, (c, ℓ) is added to $Q^{\mathsf{pk}}[i]$, and ℓ is removed from $\mathcal{B}_j^{\mathsf{pk}}[i]$.
Since the sets $\mathcal{B}_k^{\mathsf{pk}}[i]$ of other parties and $F^{\mathsf{pk}}[i]$ remain unchanged and $\mathcal{B}_j^{\mathsf{pk}}[i]$ is decreased, (3) and (4) are maintained. Next, we prove (2) is maintained. Therefore consider any $\ell \in \mathcal{B}_j^{\mathsf{pk}}[i]$: By (3) this implies $\ell \notin F^{\mathsf{pk}}[i]$ and because $F^{\mathsf{pk}}[i] = F^{\mathsf{pk}}[i+1]$, we obtain $\ell \notin F^{\mathsf{pk}}[i+1]$. Also, as we only add (c, ℓ) to $Q^{\mathsf{pk}}[i]$ but remove ℓ from $\mathcal{B}_j^{\mathsf{pk}}[i]$ while not changing any other $\mathcal{B}_k^{\mathsf{pk}}[i]$, we have $\ell \notin \mathcal{B}_i^{\mathsf{pk}}[i]$ for each party $\mathcal{P}_i$. Putting both observations together, for all parties $\mathcal{P}_i$ we obtain $\ell \notin \mathcal{B}_i^{\mathsf{pk}}[i+1] \cup F^{\mathsf{pk}}[i+1]$. Finally, by reversing (2), $\ell \in \mathcal{B}_j^{\mathsf{pk}}[i]$ implies $(\cdot, \ell) \notin Q^{\mathsf{pk}}[i]$. Hence, adding (c, ℓ) to $Q^{\mathsf{pk}}[i]$ resulting in $Q^{\mathsf{pk}}[i+1]$ does not affect (1).

□

Protocol associated to Hybrid $\mathcal{H}_{13}$. Bullet point annotations indicate in which hybrid(s) the code was introduced or changed.

- Each party ignores any message from any party that contains some session ID sid until after they have received $(\texttt{init}, \mathsf{sid})$.
- If a set is accessed although it has not been initialized, it is first initialized as $\emptyset$.
- Memory is per party and sid if not stated otherwise.

Initializiation

$^{\mathcal{H}_1}$• If $(\texttt{init}, \mathsf{sid})$ is received for the first time, set $\ell_{\mathsf{pk}}, \ell_{\mathsf{sign}} \leftarrow 1$ and $\mathcal{K}_{\mathsf{pk}}, \mathcal{K}_{\mathsf{sign}}, \mathcal{K}_{\mathsf{send}} \leftarrow \emptyset$. Set $\mathsf{rmode} \leftarrow 0$.

$\mathcal{H}_1$ • If $\mathsf{rmode} = 1$, also process the second message using the interfaces below.

Key Generation On input $(\texttt{keygen}, \mathsf{sid}, \mathcal{H})$

- If $\mathcal{P}_j \notin \mathcal{H}$, output $\perp$.

$\mathcal{H}_{3,13}$ • If $\mathsf{rmode} = 0$, compute $((\mathsf{sk}_i)_{i\in[\ell^{\mathsf{pk}}_{\mathrm{sub}}]}, \mathsf{pk}) \xleftarrow{\$} \mathsf{KeyGen}'()$. If $\mathsf{pk} \in \mathcal{K}_{\mathsf{pk}}$, set $\mathsf{rmode} \leftarrow 1$.

$\mathcal{H}_1$ • If $\mathsf{rmode} = 1$, sample $\mathsf{pk} \xleftarrow{\$} \{0,1\}^{\ell_{\mathsf{pk}}} \setminus \mathcal{K}_{\mathsf{pk}}$, set $\mathsf{sk} \leftarrow \perp$, and set $\ell^{\mathsf{pk}}_{\mathrm{sub}} \leftarrow \infty$.

$\mathcal{H}_4$ • Set $\mathcal{N}^{\mathsf{pk}}_j \leftarrow 1$ and $\mathcal{B}^{\mathsf{pk}}_j \leftarrow [\ell_{\mathrm{sub}}]$.

$\mathcal{H}_1$ • Set $\mathcal{K}_{\mathsf{pk}} \xleftarrow{\cup} \{\mathsf{pk}\}$ and increment ℓ_{pk} until $\{0,1\}^{\ell_{\mathsf{pk}}} \setminus \mathcal{K}_{\mathsf{pk}} \neq \emptyset$.

$\mathcal{H}_4$ • Store $(\texttt{key}, \mathsf{sid}, \mathcal{H}, (\mathsf{sk}_i)_{i\in[\ell_{\mathrm{sub}}]}, \mathsf{pk})$ globally.

- Output $(\texttt{pk}, \mathsf{sid}, \mathsf{pk}, \ell_{\mathrm{sub}})$.

Notify On input $(\texttt{notify}, \mathsf{sid}, \mathsf{pk}, \mathcal{P}_t)$

$\mathcal{H}_1$ • Set $\mathcal{K}_{\mathsf{pk}} \xleftarrow{\cup} \{\mathsf{pk}\}$ and increment ℓ_{pk} until $\{0,1\}^{\ell_{\mathsf{pk}}} \setminus \mathcal{K}_{\mathsf{pk}} \neq \emptyset$.

- If $(\texttt{key}, \mathsf{sid}, \mathcal{H}, \cdot, \mathsf{pk})$ exists in memory for some $\mathcal{H}$, continue, else output $\perp$.

$\mathcal{H}_4$ • If $\mathcal{P}_j \notin \mathcal{H}$, $\mathcal{P}_t \notin \mathcal{H}$, or $\mathcal{N}^{\mathsf{pk}}_j = 0$, output $\perp$.

$\mathcal{H}_4$ • Store $(\texttt{notified}, \mathsf{sid}, \mathsf{pk}, \mathcal{P}_j, \mathcal{P}_t)$ globally.

$\mathcal{H}_4$ • Set $h \leftarrow (\texttt{key}, \mathsf{pk})$, $\mu \leftarrow (0^{\mathcal{L}(i,\mathsf{pk})})_{i\in[\ell_{\mathrm{sub}}]}$ and send $(\texttt{send}, \mathsf{sid}, \mathcal{H}, \mathcal{P}_t, h, \mu)$ to $\mathcal{F}_{\mathrm{fresh}}$.

Notified $\mathcal{P}_t$ on input $(\texttt{dlvr}, \mathsf{sid}, \mathcal{H}, \mathcal{P}_j, h, \mu)$ with $h = (\texttt{key}, \mathsf{pk})$ from $\mathcal{F}_{\mathrm{fresh}}$

$\mathcal{H}_1$ • Set $\mathcal{K}_{\mathsf{pk}} \xleftarrow{\cup} \{\mathsf{pk}\}$ and increment ℓ_{pk} until $\{0,1\}^{\ell_{\mathsf{pk}}} \setminus \mathcal{K}_{\mathsf{pk}} \neq \emptyset$.

$\mathcal{H}_4$ • If $(\texttt{notified}, \mathsf{sid}, \mathsf{pk}, \mathcal{P}_j, \mathcal{P}_t)$ exists in memory, set $\mathcal{N}^{\mathsf{pk}}_t \leftarrow 1$.

Assign $\mathcal{P}_j$ on input $(\texttt{assign}, \mathsf{sid}, \mathsf{pk}, \mathcal{P}_t, \mathsf{req})$

$\mathcal{H}_1$ • Set $\mathcal{K}_{\mathsf{pk}} \xleftarrow{\cup} \{\mathsf{pk}\}$ and increment ℓ_{pk} until $\{0,1\}^{\ell_{\mathsf{pk}}} \setminus \mathcal{K}_{\mathsf{pk}} \neq \emptyset$.

- If $(\texttt{key}, \mathsf{sid}, \mathcal{H}, \cdot, \mathsf{pk})$ exists in memory for some $\mathcal{H}$, continue, else output $\perp$.
- If $\mathcal{P}_t \notin \mathcal{H}$, output $\perp$.
- If $\mathsf{req} \notin \mathcal{B}^{\mathsf{pk}}_j$, output $\perp$.
- Set $\mathcal{B}^{\mathsf{pk}}_j \xleftarrow{\setminus} \mathsf{req}$.
- Set $h \leftarrow (\texttt{assgnd}, \mathsf{pk}, \mathsf{req})$, $\mu \leftarrow \top$ and send $(\texttt{send}, \mathsf{sid}, \mathcal{H}, \mathcal{P}_t, h, \mu)$ to $\mathcal{F}_{\mathrm{fresh}}$.

Assigned $\mathcal{P}_t$ on input $(\texttt{dlvr}, \mathsf{sid}, \mathcal{H}, \mathcal{P}_j, h, \mu)$ with $h = (\texttt{assgnd}, \mathsf{pk}, \mathsf{req})$ from $\mathcal{F}_{\mathrm{fresh}}$

$\mathcal{H}_2$ • Set $\mathcal{K}_{\mathsf{pk}} \xleftarrow{\cup} \{\mathsf{pk}\}$ and increment ℓ_{pk} until $\{0,1\}^{\ell_{\mathsf{pk}}} \setminus \mathcal{K}_{\mathsf{pk}} \neq \emptyset$.

$\mathcal{H}_4$ • If $(\texttt{key}, \mathsf{sid}, \mathcal{H}, \cdot, \mathsf{pk})$ exists in memory for some $\mathcal{H}$, continue, else output $\perp$.

$\mathcal{H}_4$ • If $\mathcal{N}^{\mathsf{pk}}_t = 0$, or $\mathcal{P}_j \notin \mathcal{H}$ or $\mathcal{P}_t \notin \mathcal{H}$, output $\perp$

- Set $\mathcal{B}^{\mathsf{pk}}_t \xleftarrow{\cup} \mathsf{req}$.
- Output $(\texttt{keyset}, \mathsf{sid}, \mathsf{pk}, \mathcal{B}^{\mathsf{pk}}_t)$.

Sign On input $(\texttt{sign}, \mathsf{sid}, \mathsf{pk}, \mu, l)$

$\mathcal{H}_1$ • Set $\mathcal{K}_{\mathsf{pk}} \xleftarrow{\cup} \{\mathsf{pk}\}$ and increment ℓ_{pk} until $\{0,1\}^{\ell_{\mathsf{pk}}} \setminus \mathcal{K}_{\mathsf{pk}} \neq \emptyset$.

$\mathcal{H}_4$ • If no record $(\texttt{key}, \mathsf{sid}, \cdot, (\mathsf{sk}_i)_{i\in[\ell_{\mathrm{sub}}]}, \mathsf{pk})$ is globally stored, output $\perp$.

$\mathcal{H}_4$ • If $\mathcal{N}^{\mathsf{pk}}_j = 0$, or $l \notin \mathcal{B}^{\mathsf{pk}}_j$, output $\perp$ to $\mathcal{P}_j$.

$\mathcal{H}_{1,5,6}$ $\mathcal{H}_{7,10,12}$ • If $\mathsf{rmode} = 0$, compute $\sigma \xleftarrow{\$} \mathsf{Sign}'(\mathsf{sk}_l, \mu)$. If $(\texttt{sig}, \mathsf{sid}, \mathsf{pk}, \mu', \sigma)$ exists in memory such that $\mu \neq \mu'$ or $(\texttt{badsig}, \mathsf{sid}, \mathsf{pk}, \mu, \sigma)$ exists in memory, set $\mathsf{rmode} \leftarrow 1$.

$\mathcal{H}_1$ • If $\mathsf{rmode} = 1$, sample $\sigma \xleftarrow{\$} \{0,1\}^{\ell_{\mathrm{sign}}} \setminus \mathcal{K}_{\mathrm{sign}}$.

- Set $\mathcal{B}^{\mathsf{pk}}_j \xleftarrow{\setminus} \{l\}$.

$\mathcal{H}_1$ • Set $\mathcal{K}_{\mathrm{sign}} \xleftarrow{\cup} \{\sigma\}$ and increment ℓ_{sign} until $\{0,1\}^{\ell_{\mathrm{sign}}} \setminus \mathcal{K}_{\mathrm{sign}} \neq \emptyset$.

$\mathcal{H}_2$ • Store $(\texttt{sig}, \mathsf{sid}, \mathsf{pk}, \mu, \sigma)$ in memory and output $(\texttt{signature}, \mathsf{sid}, \mathsf{pk}, \mu, \sigma)$ to $\mathcal{P}_j$.

Verify On input $(\texttt{vrfy}, \mathsf{sid}, \mathsf{pk}, \mu, \sigma)$

$\mathcal{H}_1$ • Set $\mathcal{K}_{\mathsf{pk}} \xleftarrow{\cup} \{\mathsf{pk}\}$ and increment ℓ_{pk} until $\{0,1\}^{\ell_{\mathsf{pk}}} \setminus \mathcal{K}_{\mathsf{pk}} \neq \emptyset$.

$\mathcal{H}_8$ • If $(\texttt{sig}, \mathsf{sid}, \mathsf{pk}, \mu, \sigma)$ exists in memory, set $b \leftarrow 1$.

$\mathcal{H}_9$ • Else, if $(\texttt{badsig}, \mathsf{sid}, \mathsf{pk}, \mu, \sigma)$ exists in memory, set $b \leftarrow 0$.

$\mathcal{H}_{11}$ • Else, if $(\texttt{key}, \mathsf{sid}, \cdot, \cdot, \mathsf{pk})$ exists in memory, set $b \leftarrow 0$.

$\mathcal{H}_1$ • Else, if $\mathsf{rmode} = 1$, set $b \leftarrow 0$.

$\mathcal{H}_1$ • Else, if $\mathsf{rmode} = 0$, set $b \xleftarrow{\$} \mathsf{Vrfy}'(\mathsf{pk}, \mu, \sigma)$.

$\mathcal{H}_9$ • If $b = 0$ and $(\texttt{badsig}, \mathsf{sid}, \mathsf{pk}, \mu, \sigma)$ does not exist in memory, store $(\texttt{badsig}, \mathsf{sid}, \mathsf{pk}, \mu, \sigma)$ in memory.

$\mathcal{H}_9$ • If $b = 1$ and no $(\texttt{sig}, \mathsf{sid}, \mathsf{pk}, \mu, \sigma)$ exists in memory, store $(\texttt{sig}, \mathsf{sid}, \mathsf{pk}, \mu, \sigma)$ in memory.

$\mathcal{H}_1$ • Set $\mathcal{K}_{\mathrm{sign}} \xleftarrow{\cup} \{\sigma\}$ and increment ℓ_{sign} until $\{0,1\}^{\ell_{\mathrm{sign}}} \setminus \mathcal{K}_{\mathrm{sign}} \neq \emptyset$.

- Output $(\texttt{verified}, \mathsf{sid}, \mathsf{pk}, \mu, \sigma, b)$.